

Efficient Memory Management and Access Mechanisms for CPU-PIM Co-Execution Systems

Yilong Zhao

Shanghai Jiao Tong University

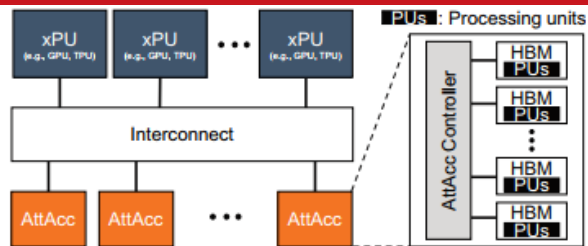
June 21, 2026

饮水思源 · 爱国荣校



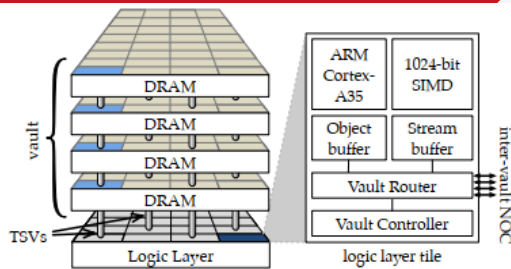
DRAM-based Processing-in-Memory Has Proven Its Potential

Researches on Domain-Specific PIMs



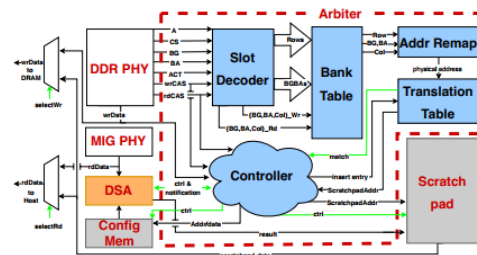
Large Language Models

[ASPLOS'24] [ASPLOS'25] [ISSCC'21]
[ISSCC'22] [HPCA'25]



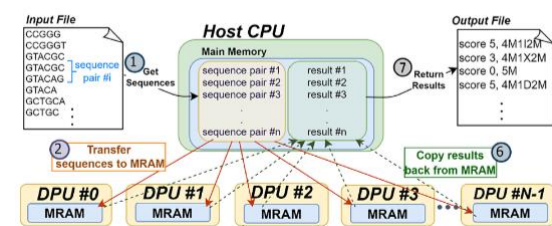
Database

[ISCA'17][PACMMOD'23]
[VLDB'23][MICRO'20][ICDE'22]



Internet Protocol

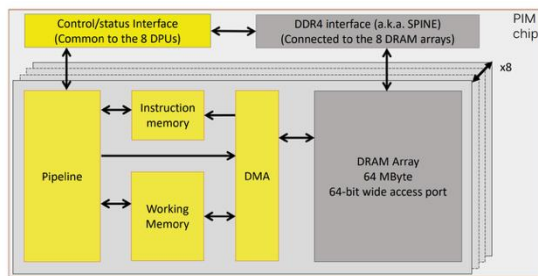
[HPCA'24] [MICRO'23]



Genomic Matching

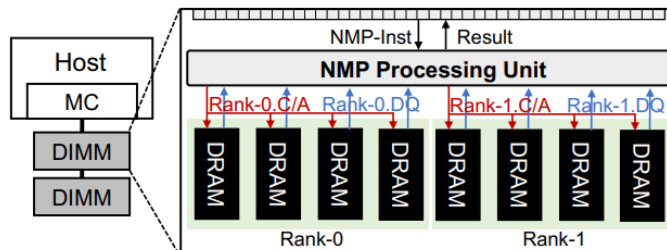
[Bioinformatics'23]

Commercial PIMs Architectures



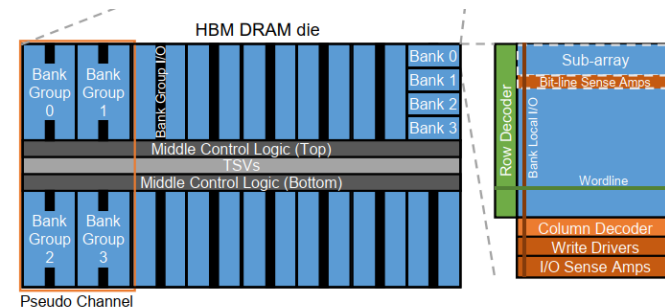
UPMEM

[HCS'19] (DDR, Bank)



AxDIMM

[MICRO'21] (DDR, DIMM)



HBM-PIM

[ISCA'21] (HBM, Bank)

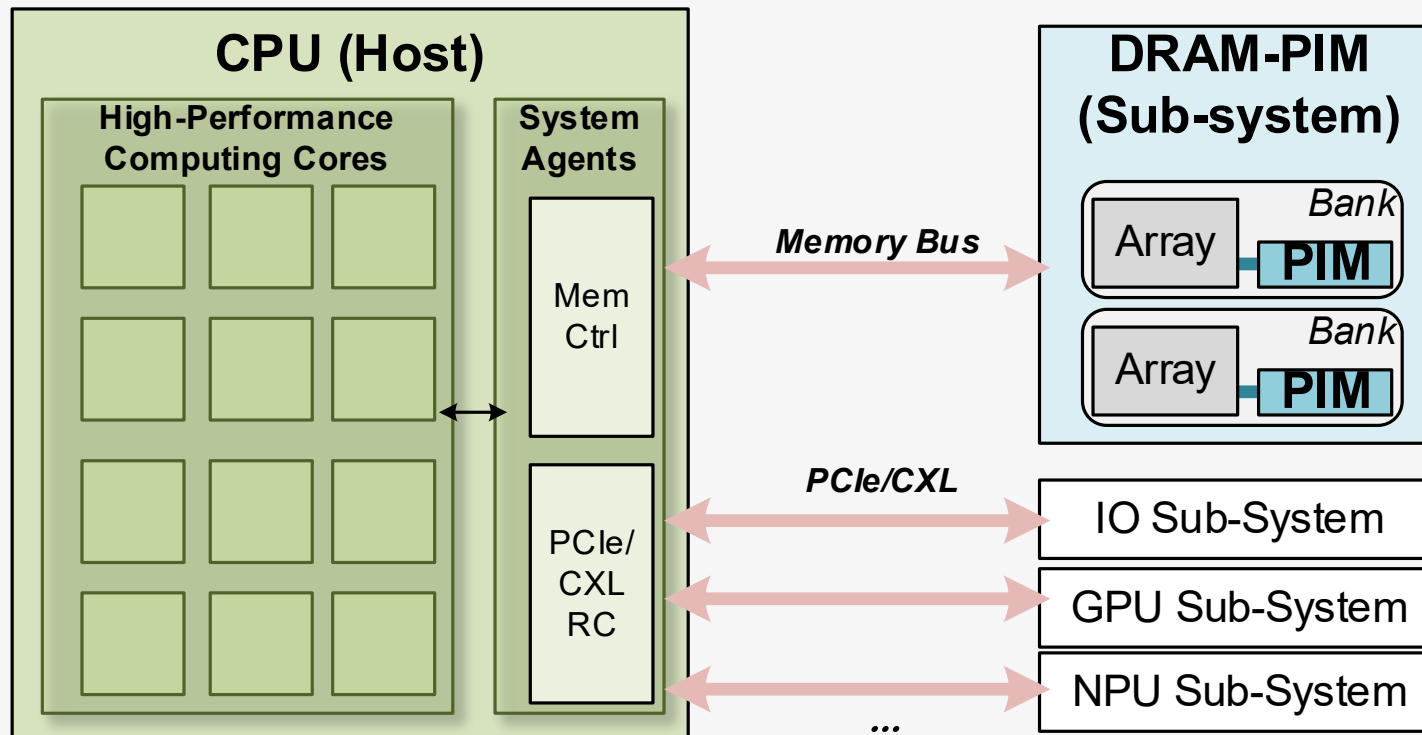


PIM Integration: Coexist with Mature CPU system



1 Global System
Orchestration

2 **Compute-**
intensive Kernels



**Data-intensive
Kernels
(Locally)**

CPU-PIM Synergy: Coexistence and Complementarity



[ISCA'24] UM-PIM: DRAM-based PIM with Uniform and Shared Memory Space

Yilong Zhao, Mingyu Gao, Fangxin Liu*, Yiwei Hu, Zongwu Wang, Han Lin, Ji Li, He Xian, Hanlin Dong, Tao Yang, Naifeng Jing, Xiaoyao Liang, Li Jiang*

Shanghai Jiao Tong University, Tsinghua University and Huawei

[ASPLOS'25] PUSHtap: PIM-based In-Memory HTAP with Unified Data Storage Format

Yilong Zhao, Mingyu Gao, Huanchen Zhang, Fangxin Liu*, Gongye Chen, He Xian, Haibing Guan, and Li Jiang*

Shanghai Jiao Tong University, Tsinghua University

[ISCA'26] COSM: A Cooperative Scheduling Framework for Concurrent PIM and CPU Execution on Mobile Devices

Yilong Zhao=, Fangxin Liu=, Onur Mutlu, Mingyu Gao, Jian Liu, Haibing Guan, and Li Jiang*

Shanghai Jiao Tong University, ETH Zurich, Tsinghua University and Beihang University

Monday, Jun 29 10:15 – 11:55 AM
Ballroom A, Session 1A



[ISCA'24] UM-PIM: DRAM-based PIM with Uniform and Shared Memory Space

Yilong Zhao, Mingyu Gao, Fangxin Liu*, Yiwei Hu, Zongwu Wang, Han Lin, Ji Li, He Xian, Hanlin Dong, Tao Yang, Naifeng Jing, Xiaoyao Liang, Li Jiang*

Shanghai Jiao Tong University, Tsinghua University and Huawei

[ASPLOS'25] PUSHtap: PIM-based In-Memory HTAP with Unified Data Storage Format

Yilong Zhao, Mingyu Gao, Huanchen Zhang, Fangxin Liu*, Gongye Chen, He Xian, Haibing Guan, and Li Jiang*

Shanghai Jiao Tong University, Tsinghua University

[ISCA'26] COSM: A Cooperative Scheduling Framework for Concurrent PIM and CPU Execution on Mobile Devices

Yilong Zhao=, Fangxin Liu=, Onur Mutlu, Mingyu Gao, Jian Liu, Haibing Guan, and Li Jiang*

Shanghai Jiao Tong University, ETH Zurich, Tsinghua University and Beihang University

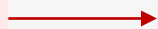
Monday, Jun 29 10:15 – 11:55 AM
Ballroom A, Session 1A



- **Problem** : data layout conflict -- **CPU memory interleaving vs. PIM locality**
- **Drawback of current solution:**
 - Isolated memory space (CPU space + PIM space) → Additional data movement
- **Insight:** One **uniform memory space**, with CPU & PIM co-exist in it

- **Challenges → Solutions :**

1 Co-existence of CPU/PIM Pages



Chunk-based memory management (SW)

2 CPU/PIM Page Layouts



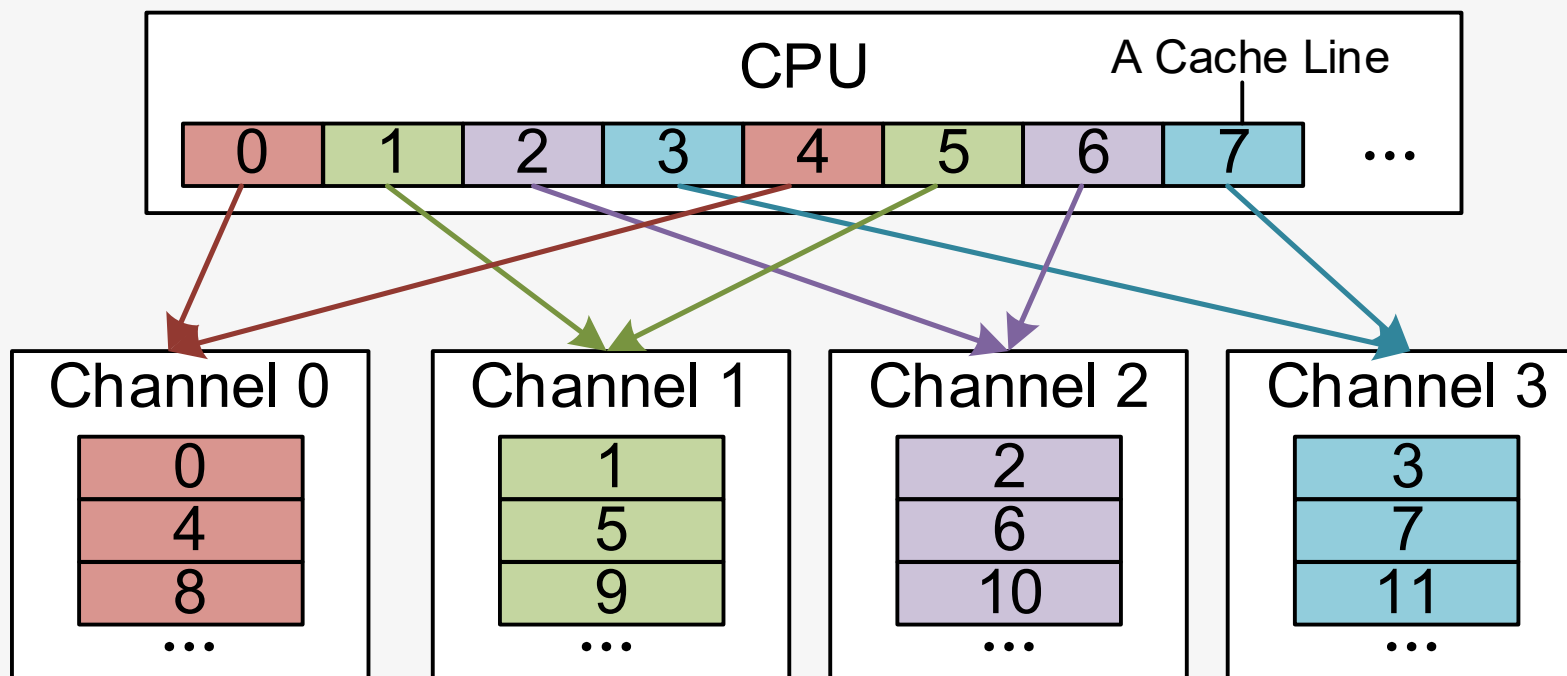
Dynamic Memory Mapping + Data Re-layout Module
(HW on DRAM side)

- **Evaluation** : Gem5+Ramulator2 simulation | 8-Channel DDR4 | PRIM benchmark
- **Results** : 1.96× peak PIM speedup (to baseline with isolated memory space) | < 0.1% CPU degradation



Memory interleaving in conventional CPU systems: increase bandwidth

Adjacent data blocks stored in different modules,
→ Can be accessed in parallel/with pipeline

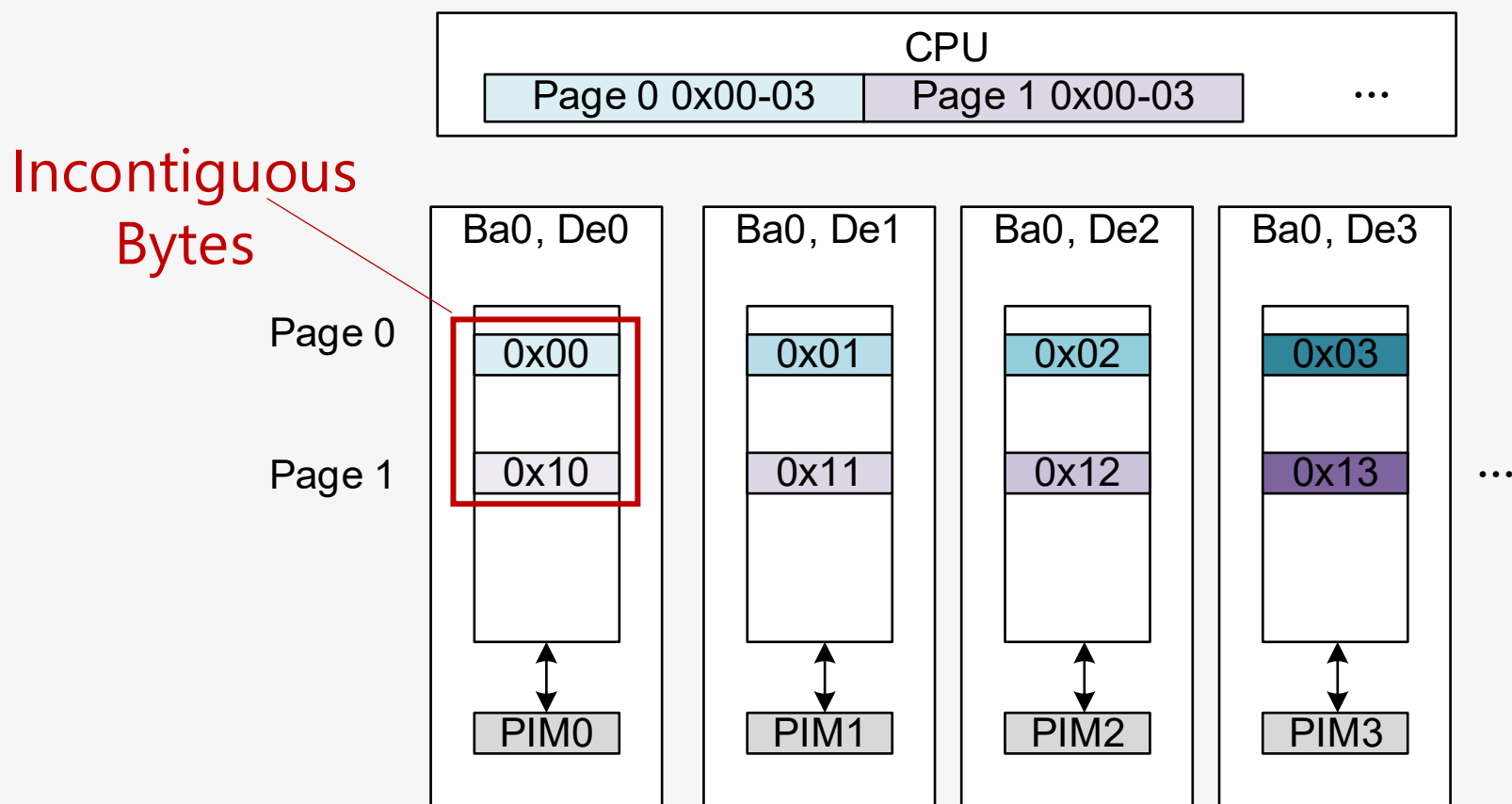


An example of interleaving (on channel level)



Limitation: Poor Locality of PIM Workload

- 1 Contiguous virtual pages are mapped to non-contiguous physical frames.
 - 2 Physical pages are interleaved to different sub-modules
- **PIM only visible to short contiguous data block**





Limitation: Poor Locality of PIM Workload (2)

Inefficiency caused by poor locality: n pages needs n offloads

Offload:

Transfer start address,
context...

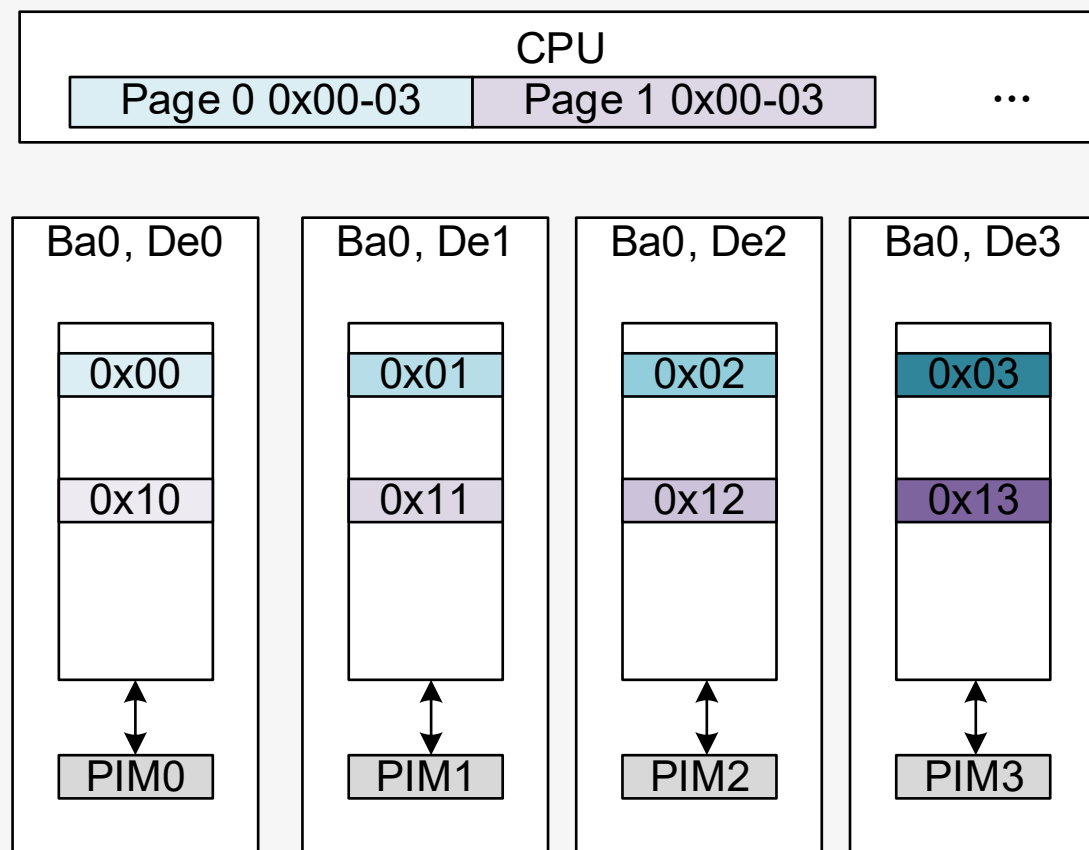
~50us/
offload

Task:

Process 1-byte data

~30ns/
task

→ Offload tiny PIM workloads
many times, each time incurs large
overhead ✕





An Ideal Data Layout for PIM



If data is non-interleaved, the same task only need to offload once!

Offload:

Transfer start address,
context...

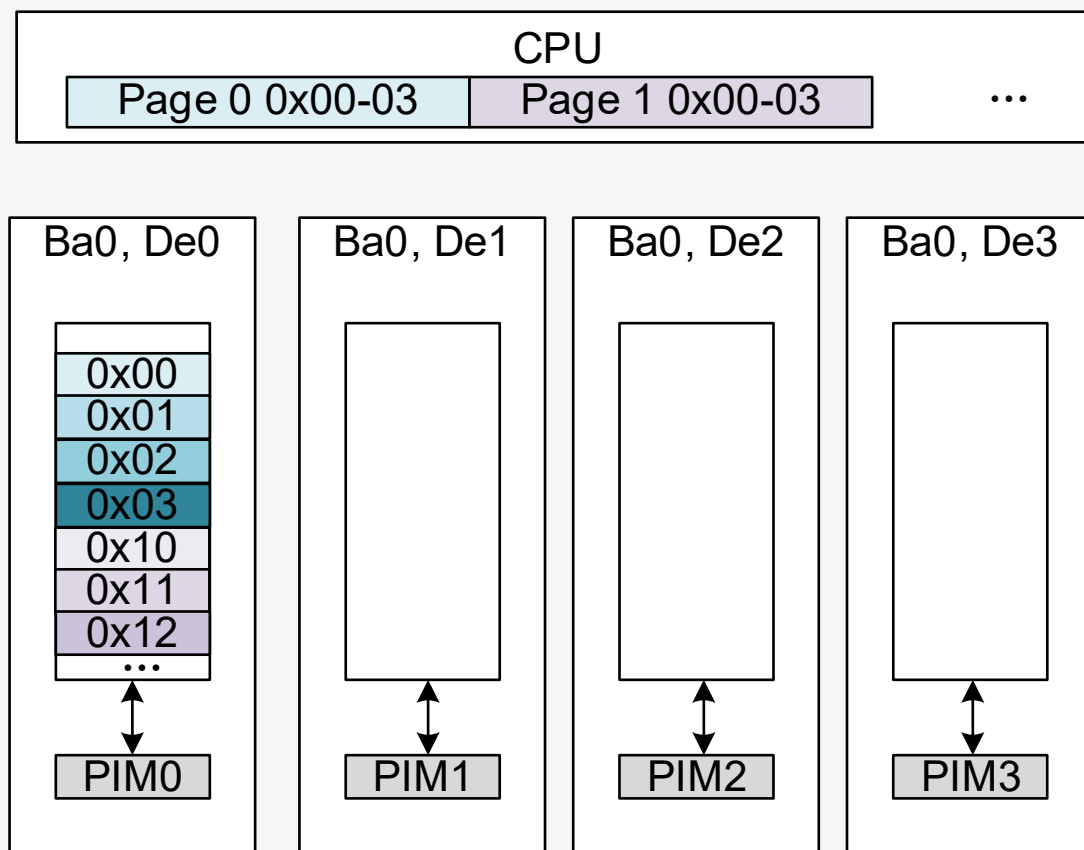
~50us/
offload

Task:

PIM automatically process
subsequent data sequentially

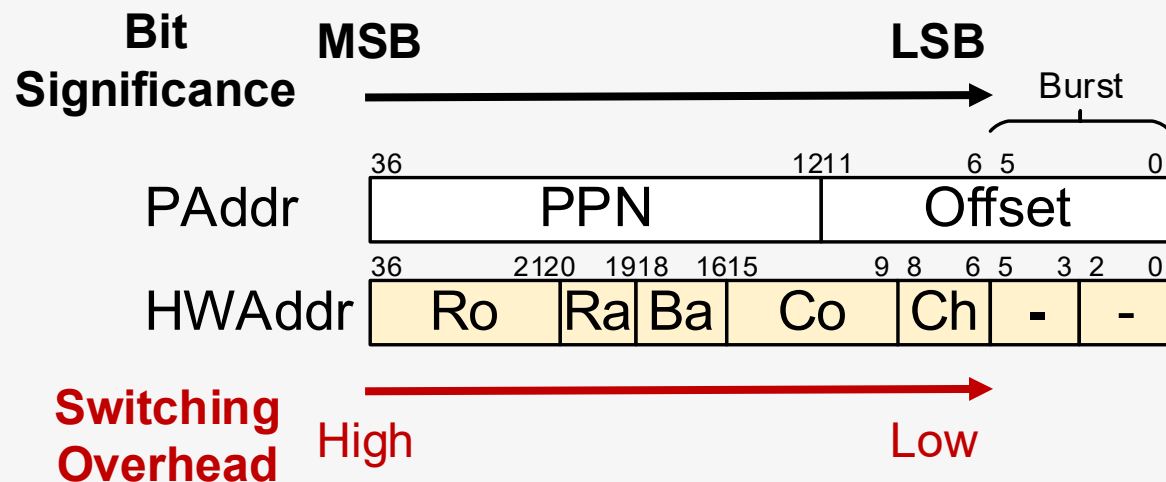
>1s/
task

→ One offload followed by a
long task, amortizing offload
overhead ✓

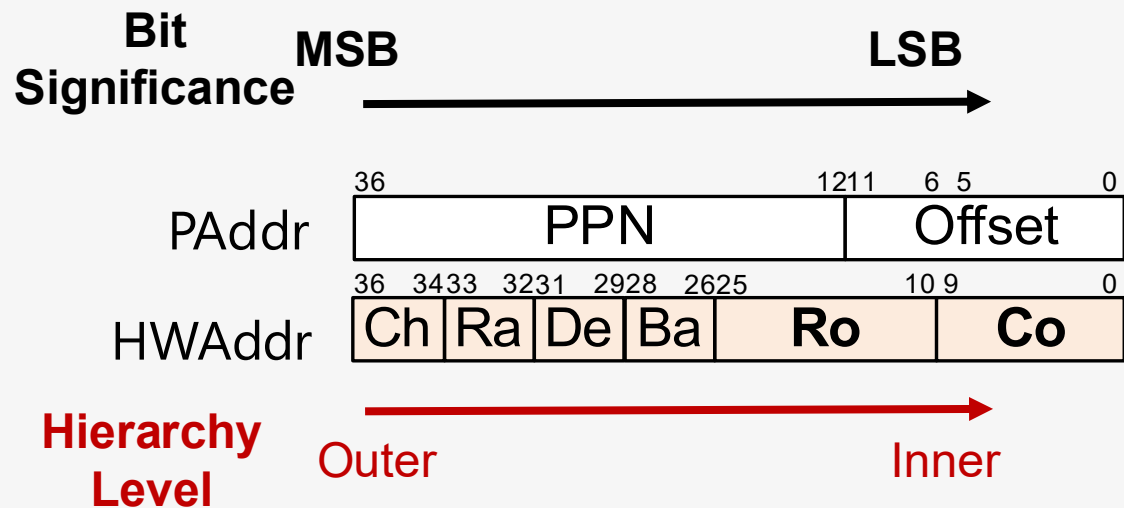




Different Layout Requires Different Address Mappings



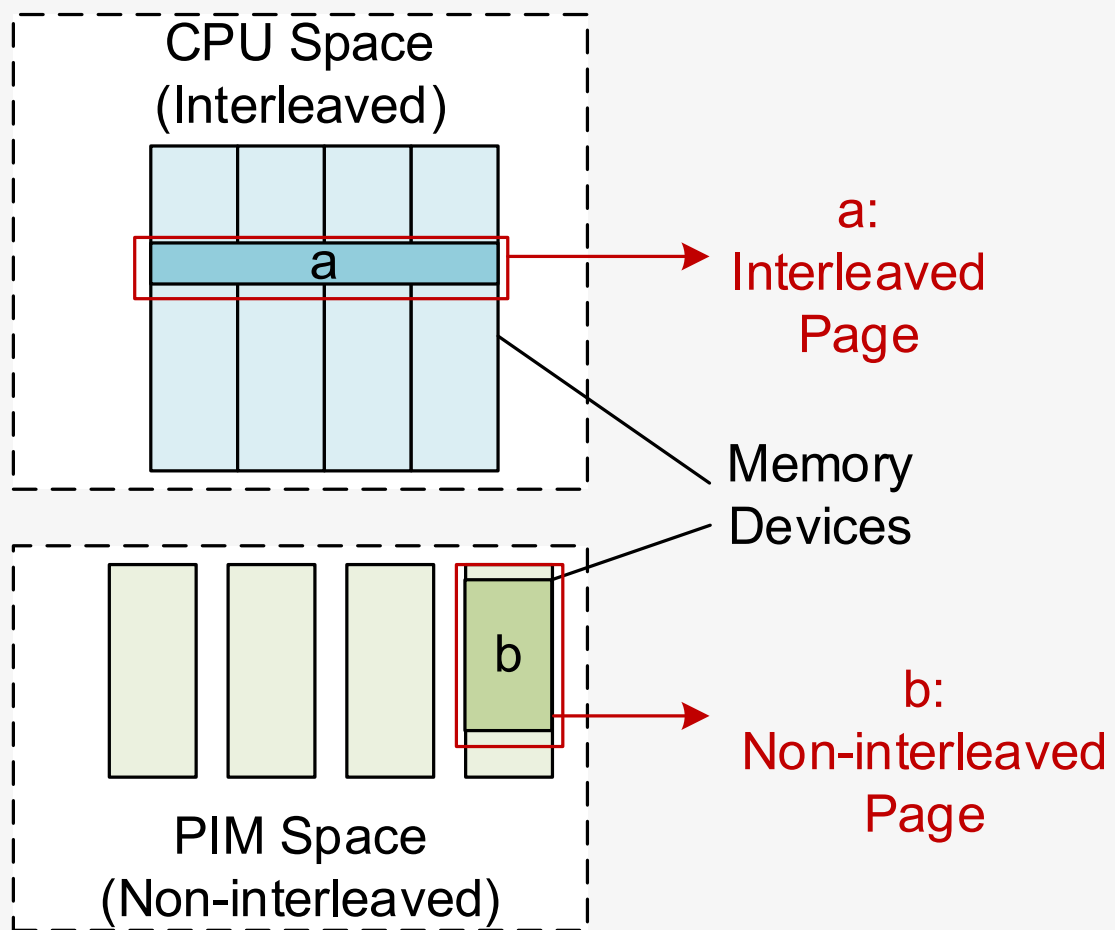
CPU's interleaving layout:
Mapping according to
switching overhead



PIM's non-interleaving layout:
Mapping according to
hierarchy level



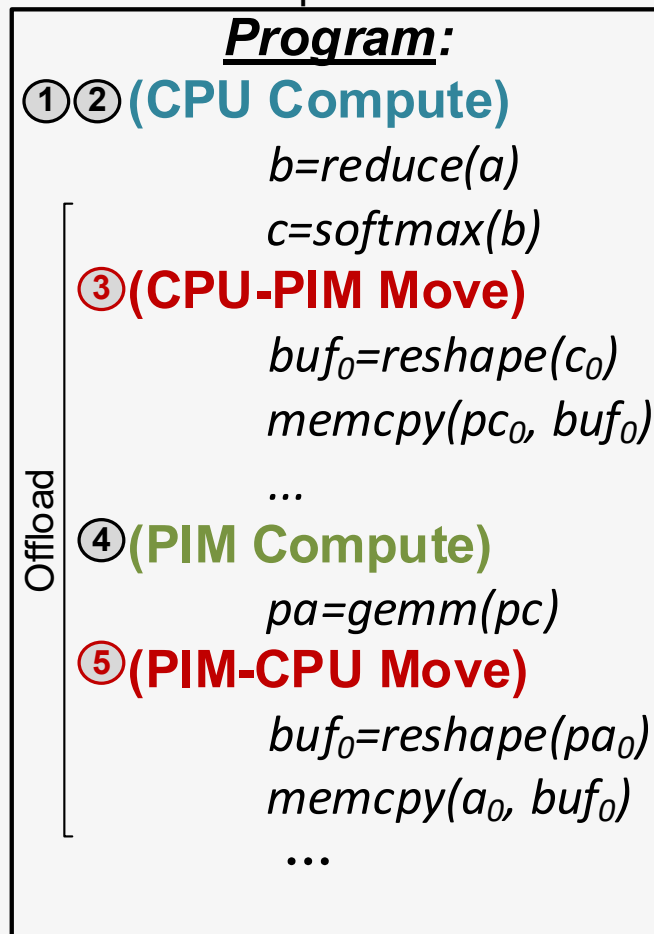
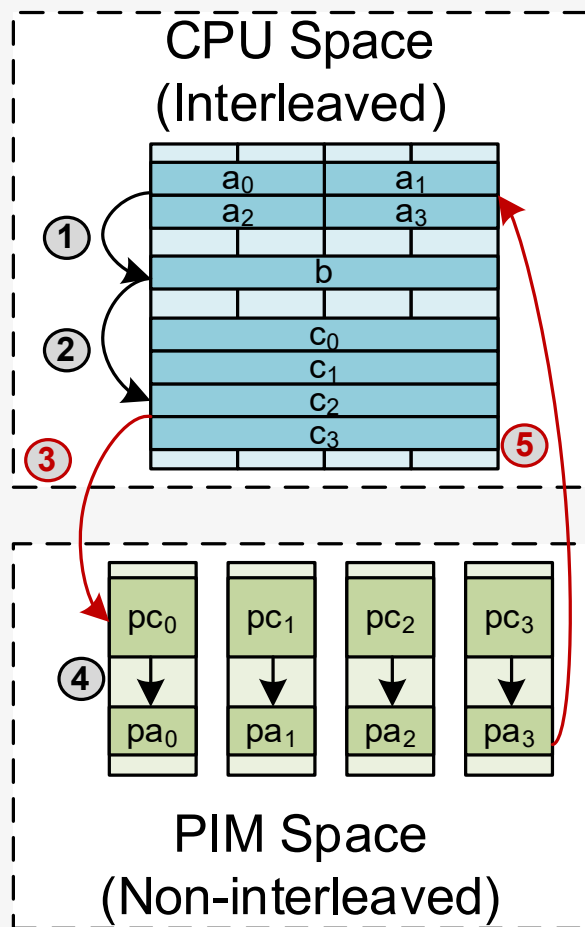
Existing Solution – Isolated Memory Space





Computation under Isolated Memory Space:

Example: MLP



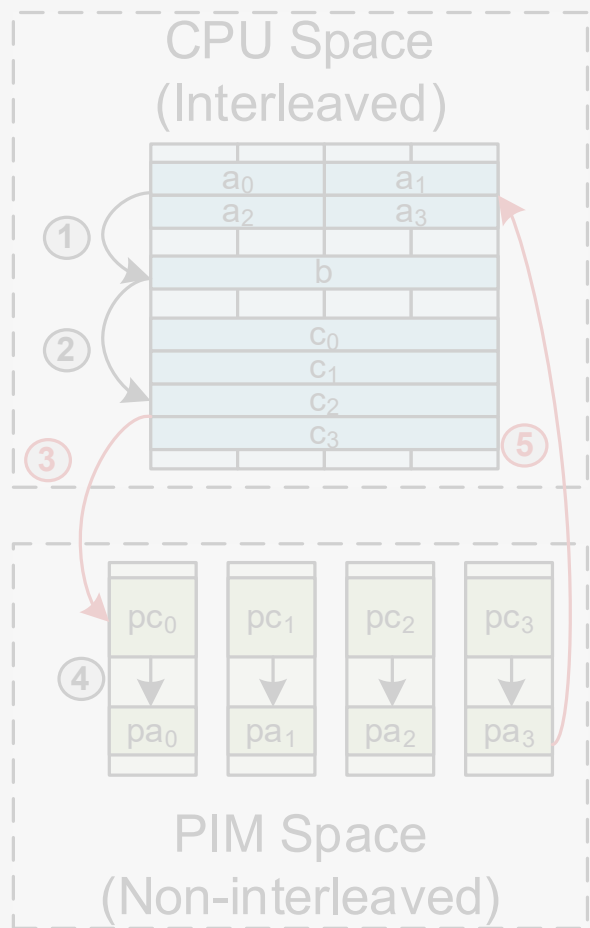


Existing Solution – Isolated Memory Space



Computation under Isolated Memory Space:

Example: MLP



Program:

①② (CPU Compute)
 $b = \text{reduce}(a)$
 $c = \text{softmax}(b)$

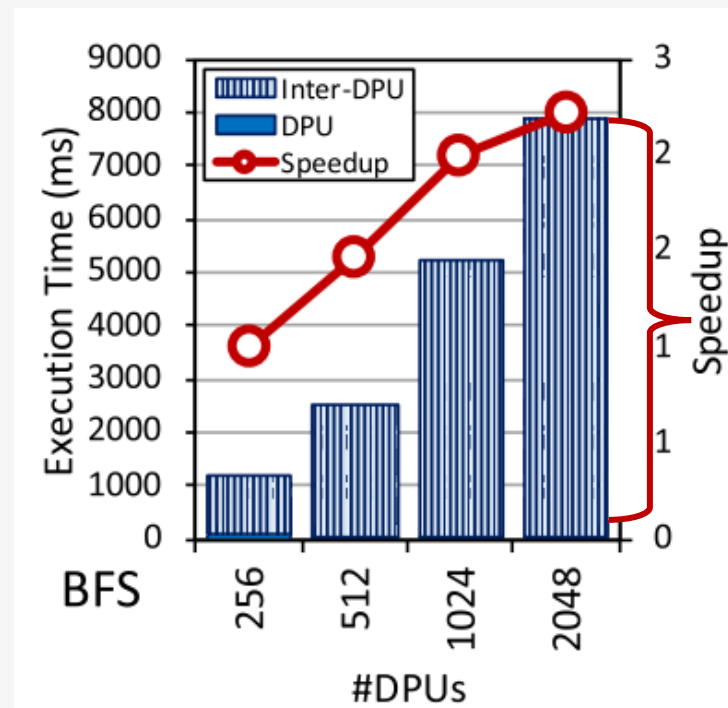
③ (CPU-PIM Move)
 $\text{buf}_0 = \text{reshape}(c_0)$
 $\text{memcpy}(pc_0, \text{buf}_0)$
...

④ (PIM Compute)
 $pa = \text{gemm}(pc)$

⑤ (PIM-CPU Move)
 $\text{buf}_0 = \text{reshape}(pa_0)$
 $\text{memcpy}(a_0, \text{buf}_0)$
...

Offload

3, 5: Additional data copy!



Over-head

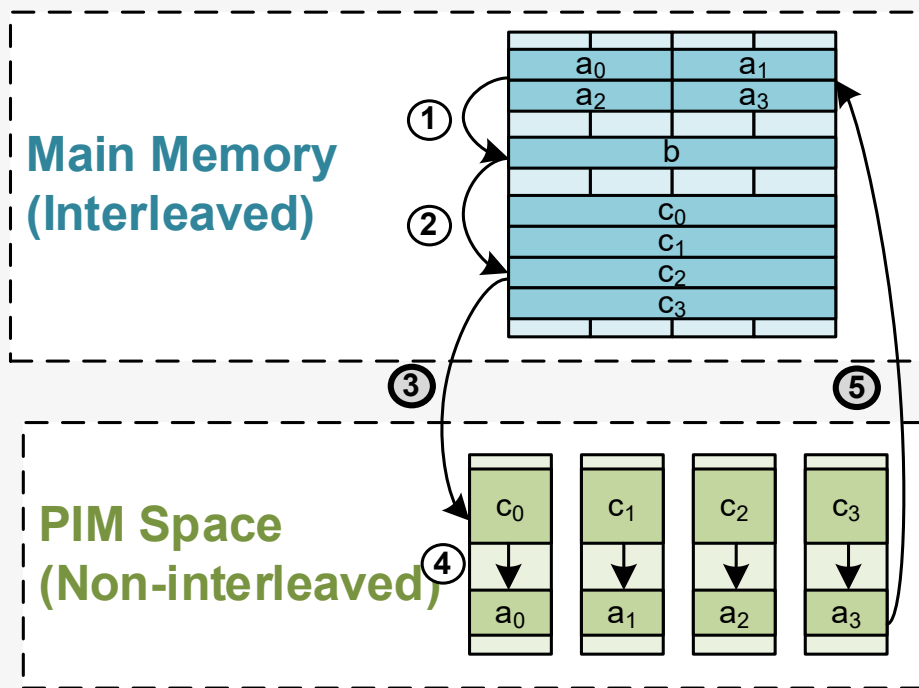
Gómez-Luna et.al. 2022



UM-PIM Contribution

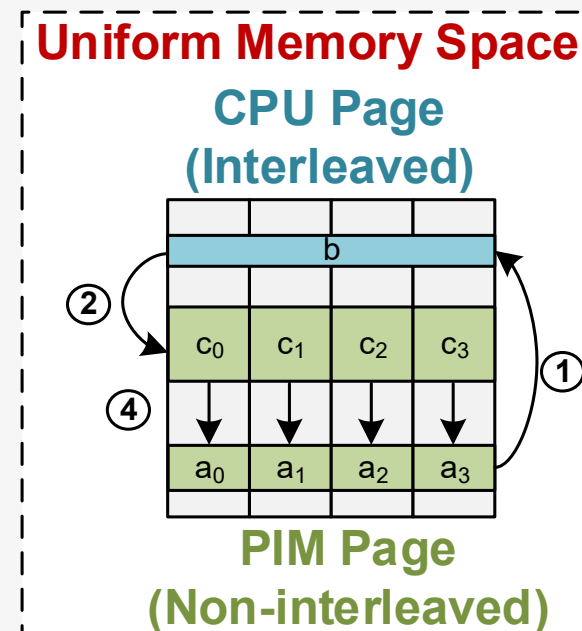
Key insight: A Uniform Memory Space with CPU & PIM Pages (different layout) co-existing in the space

- No Data Transfer Overhead
- Satisfy CPU and PIM's different data-layouts



Current PIM System

No Need for
3 & 5 Movement



UM-PIM



🔑 **Key insight:** A Uniform Memory Space with CPU & PIM Pages (different layout) co-existing in the space

Challenge 1: Co-existence

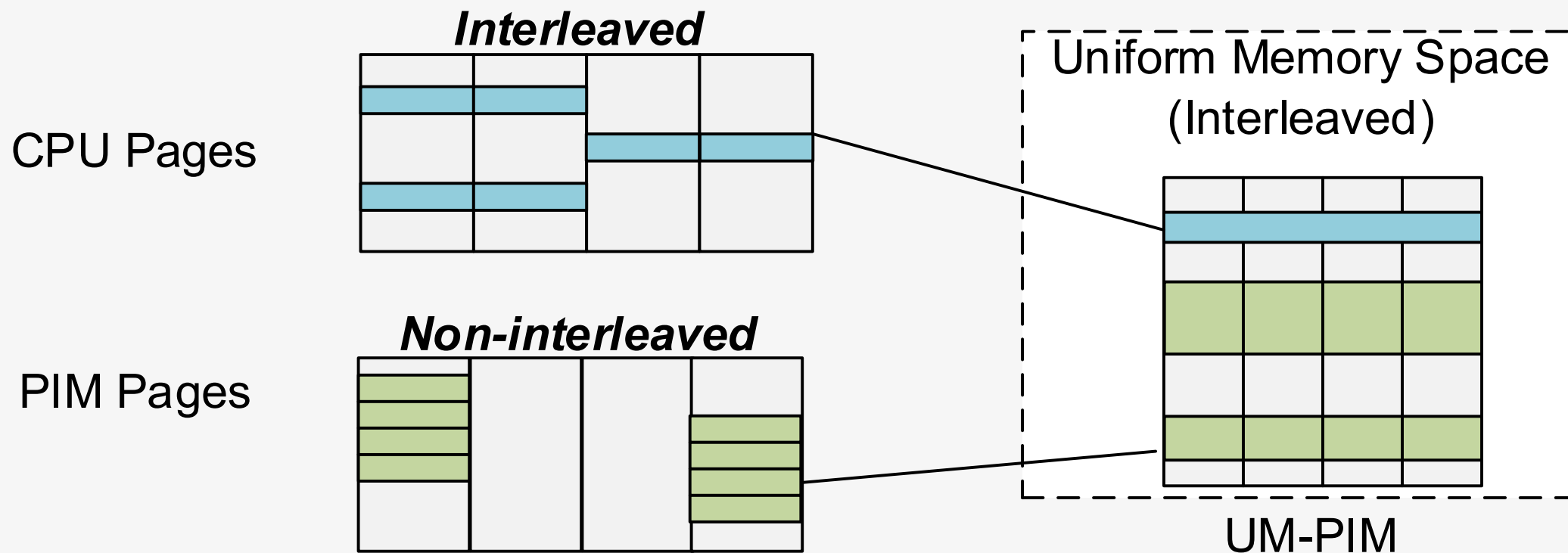
Challenge 2: Different Layout



UM-PIM Motivation: A Uniform Memory Space



🔴 Challenge 1: co-existence of CPU & PIM's pages under a single VM space



How to maintain PIM pages in the interleaved space?
Solution A. Chunk-based Management

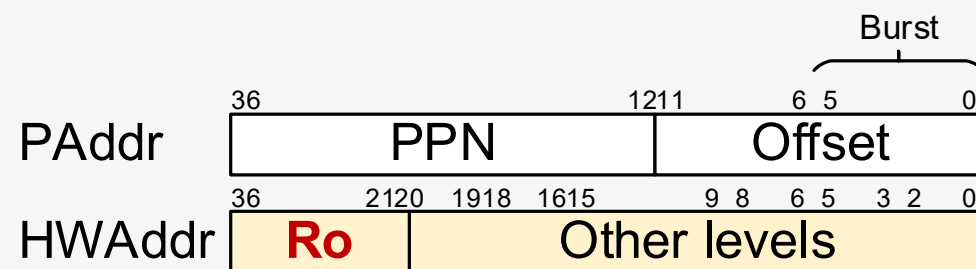
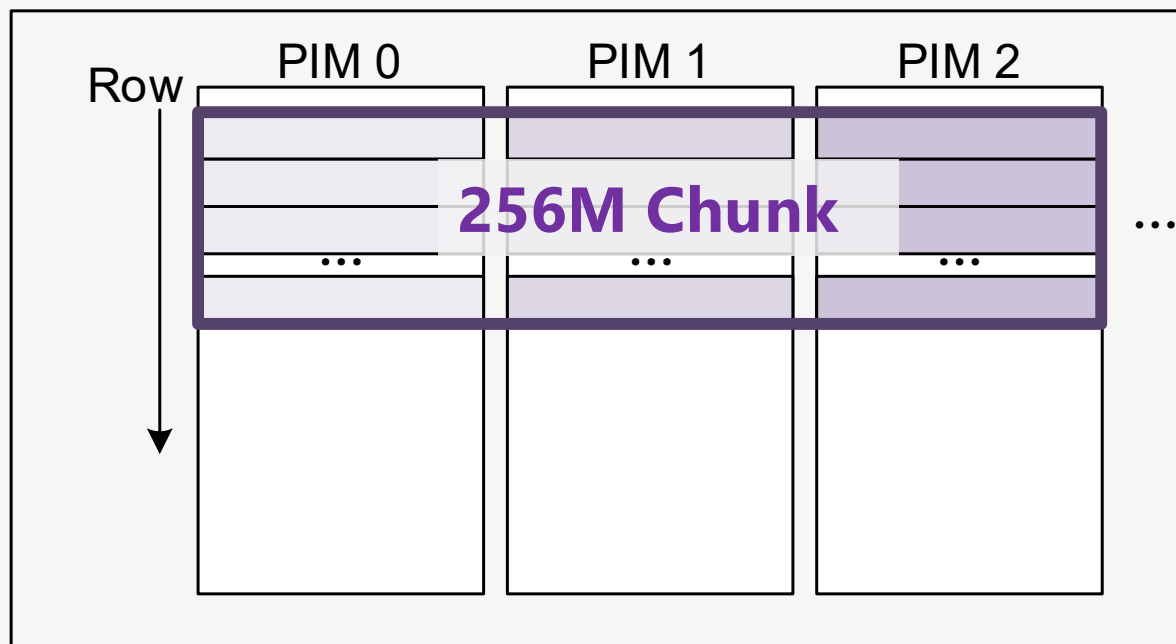


A. Memory Management – CPU Allocate PIM Page ①

System Config:
8 Channels * 4 Ranks *
16 Banks * 8 Devices

PIM Pages: Chunk-based Management

- Allocate a chunk through **transparent huge page (THP)** (e.g., 256M)



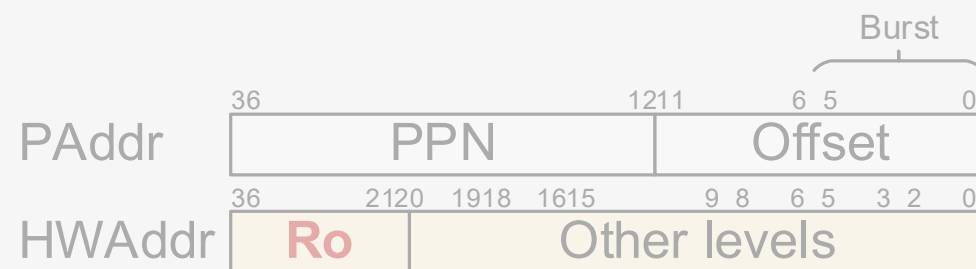
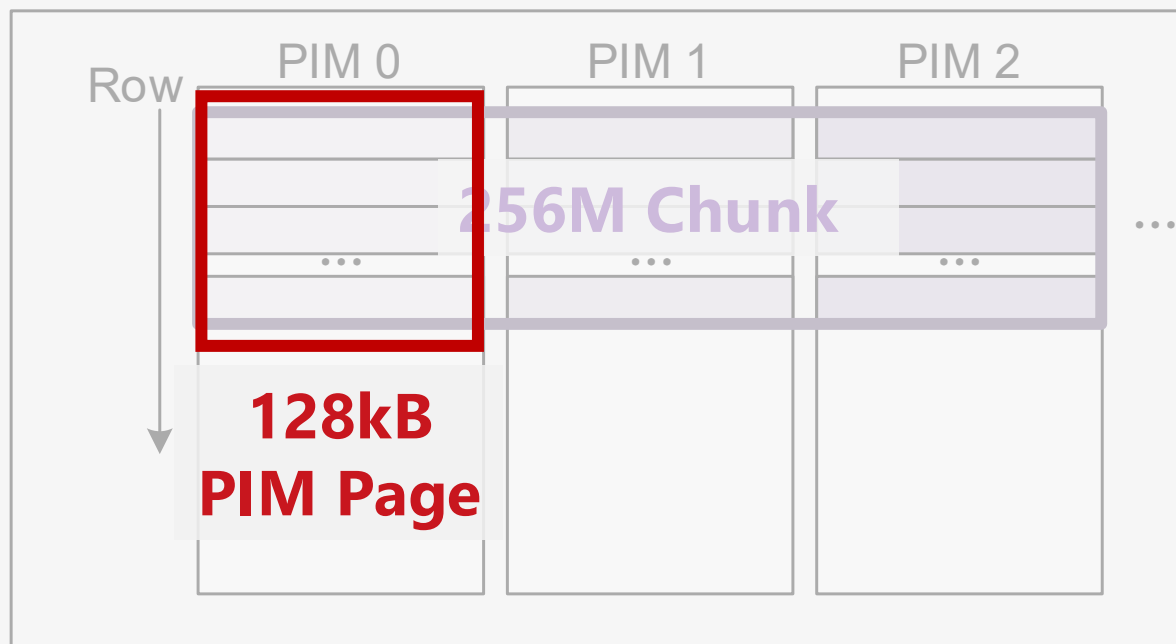


A. Memory Management – CPU Allocate PIM Page ①

System Config:
8 Channels * 4 Ranks *
16 Banks * 8 Devices

PIM Pages: Chunk-based Management

- Allocate a chunk through **transparent huge page (THP)** (e.g., 256M)
- Chunk evenly distributed on every PIMs (with *row* mapped to higher-order bits)



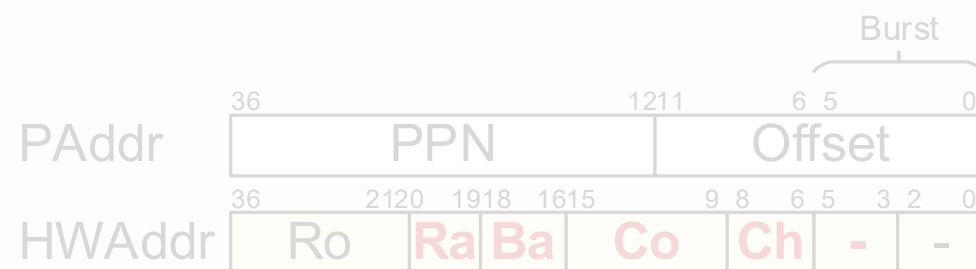
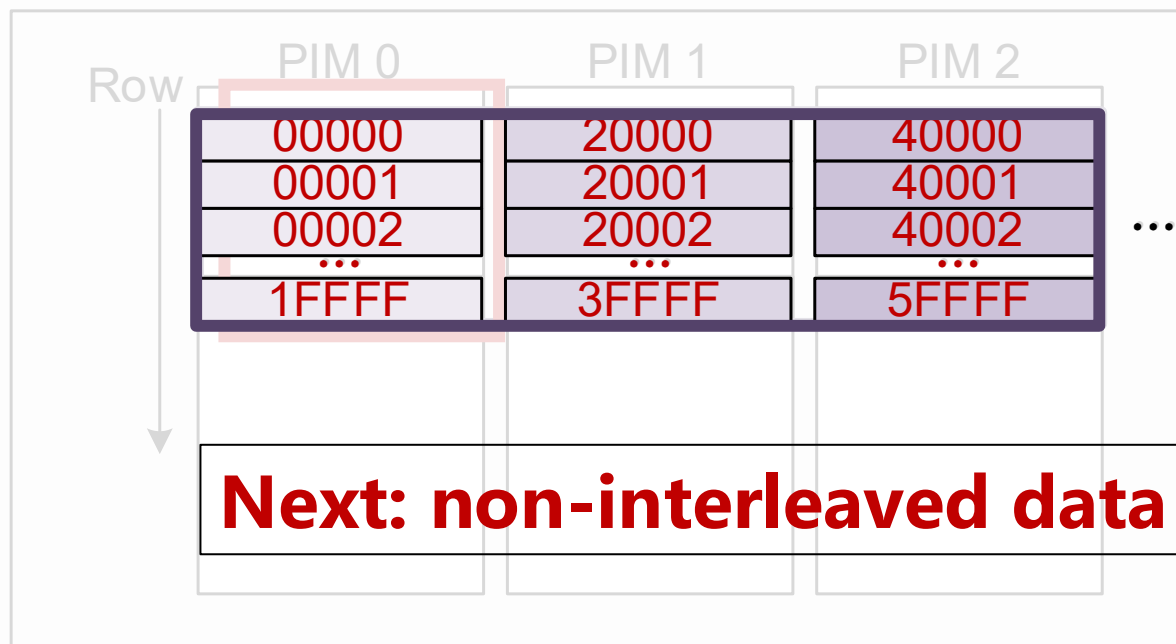


A. Memory Management – CPU Allocate PIM Page ①

System Config:
8 Channels * 4 Ranks *
16 Banks * 8 Devices

⊙ PIM Pages: Huge-page-based Management

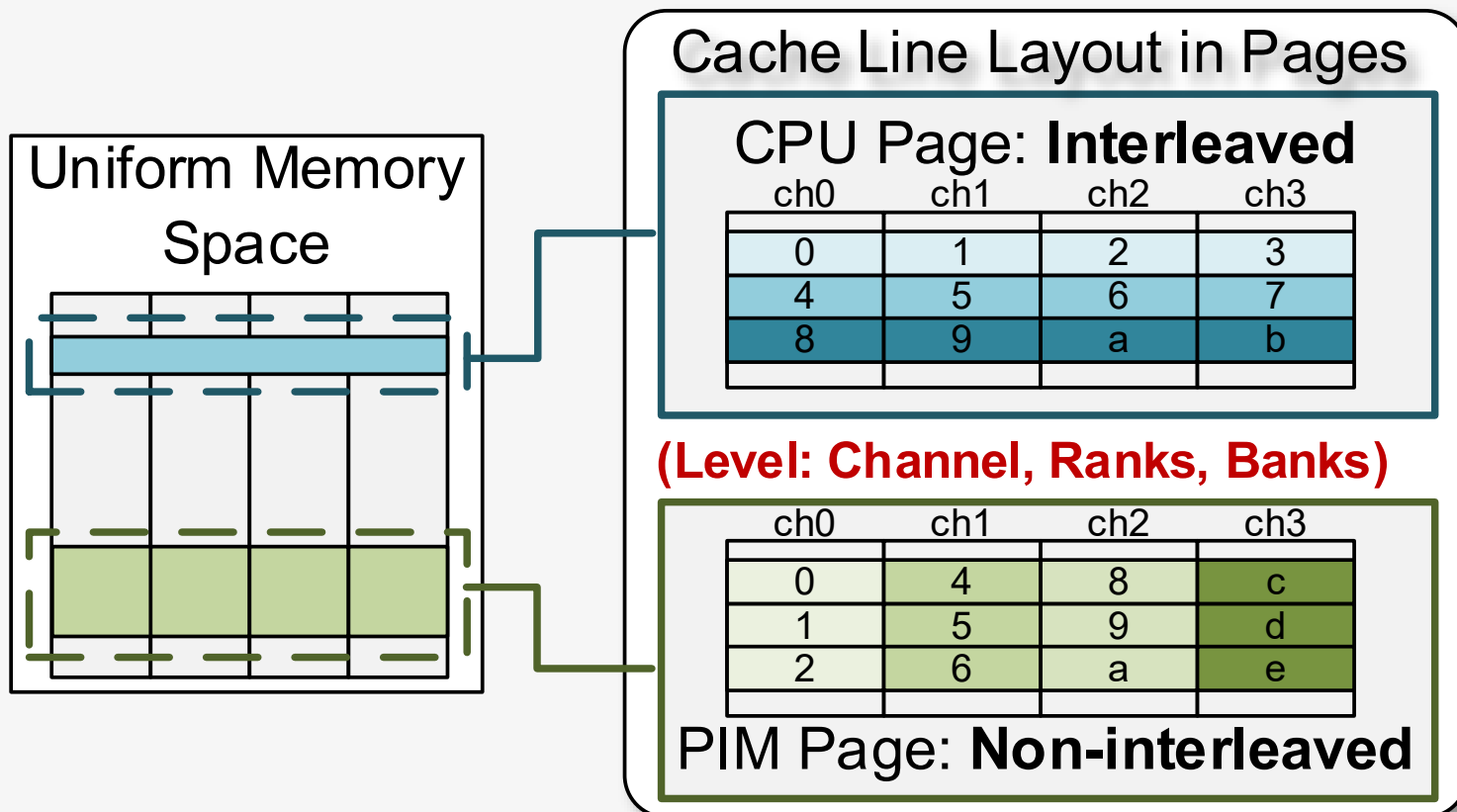
- Allocate a chunk through **transparent huge page (THP)** (e.g., 256M)
- Chunk evenly distributed on every PIMs (Row is mapped to higher-order bits)





UM-PIM Motivation: A Uniform Memory Space

- Challenge 1: co-existence of CPU & PIM's pages under a single VM space
- Challenge 2: implementing two different data layouts with low cost



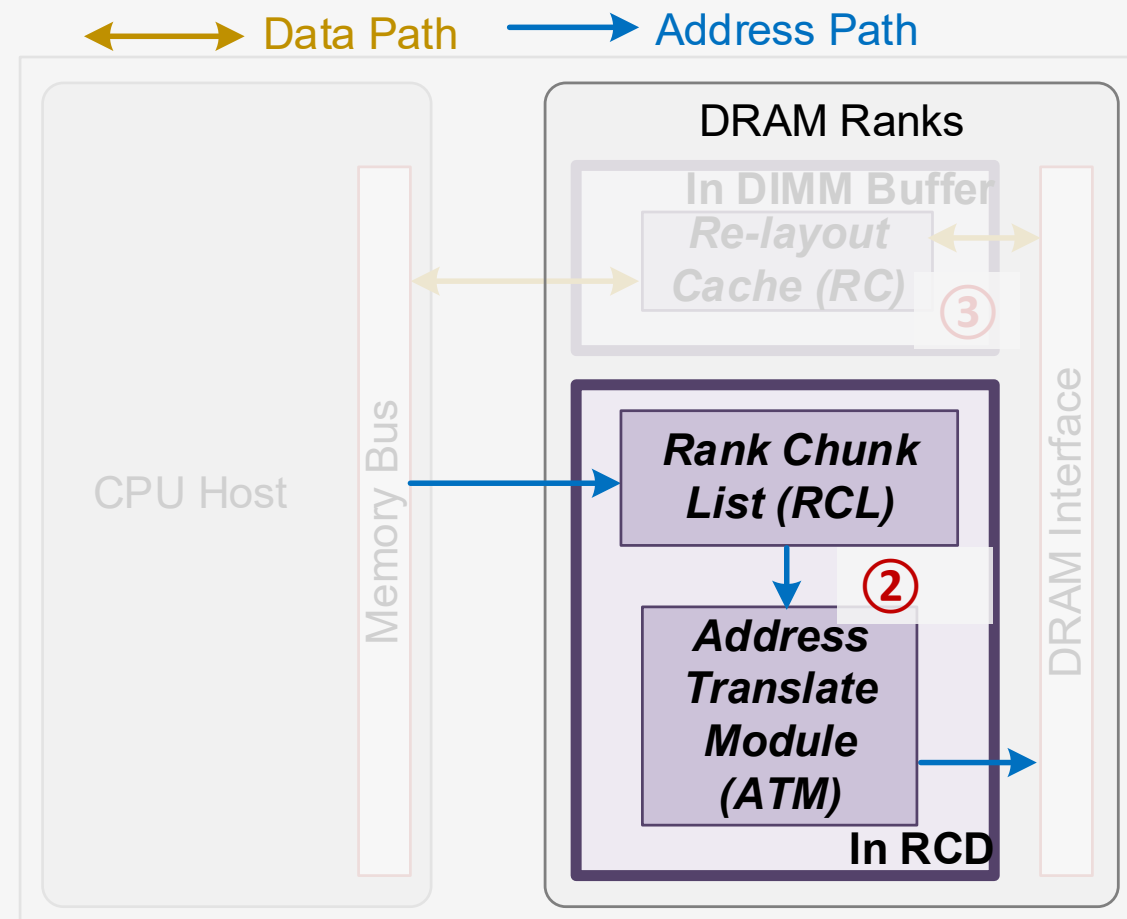
Addressable:
B.1 Dynamic Address Mapping



B. Data Layout -- Overview

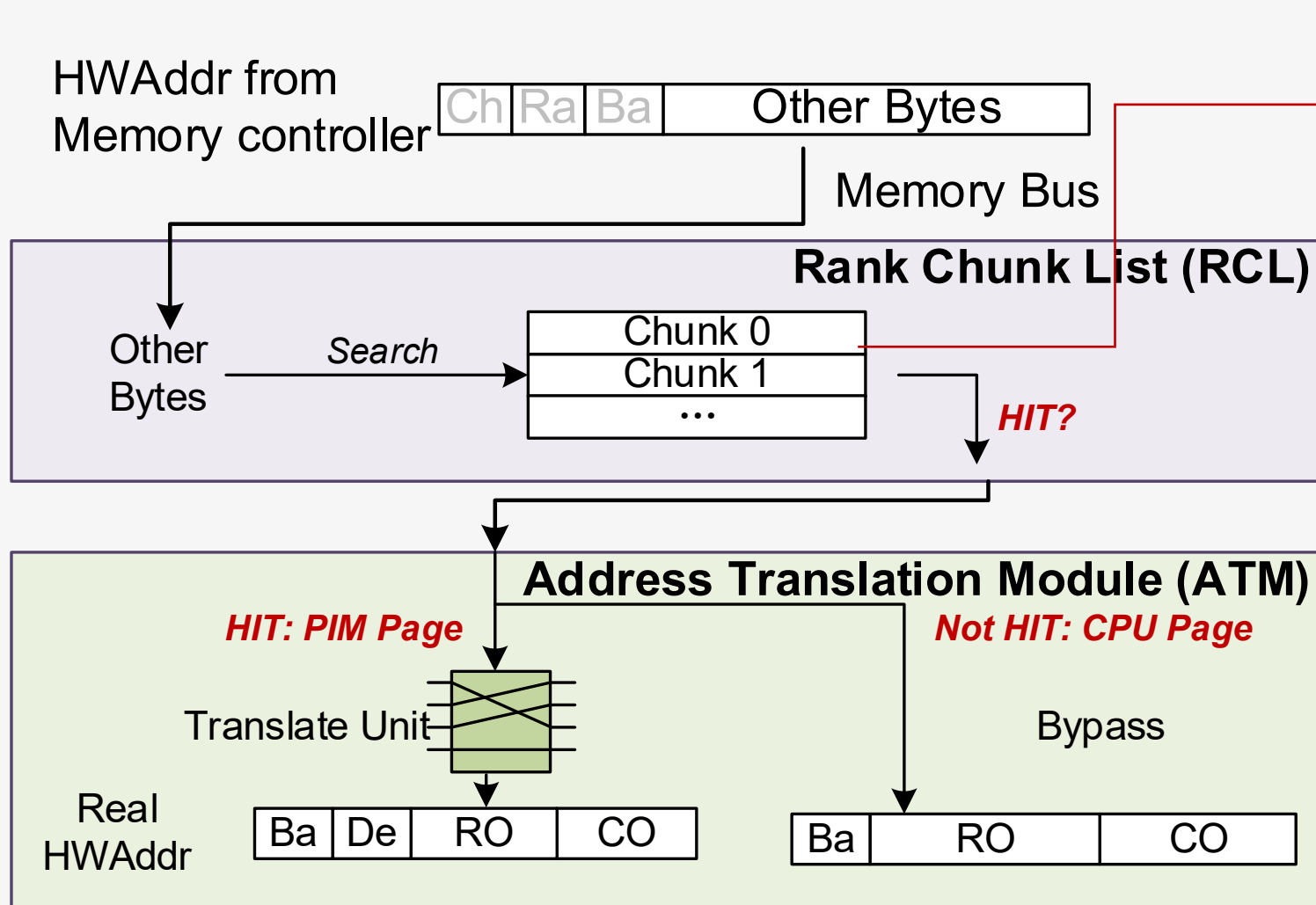


- ① B.1 Cache line layout inside page
 - Dynamic Address Mapping ②
 - DIMM-level Hardware (ATM&RCL)
- ② B.2 Byte layout inside Cache Line
 - Data re-layout ③
 - PIM-side hardware (RC)





B. Data Layout — Address Mapping (Below Bank) ②



Start addresses of allocated chunks (Set when the chunks are allocated)

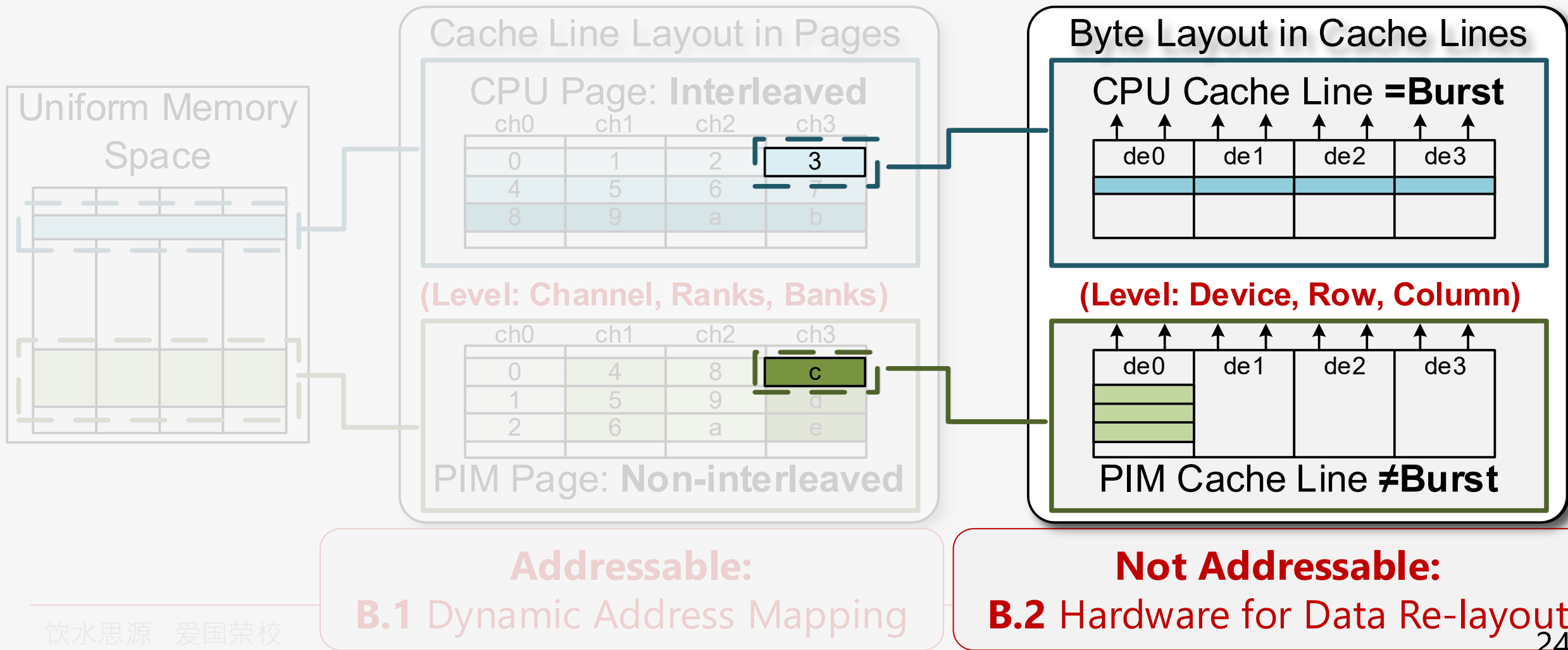
RCL:
Check **whether the address belongs** to a PIM page

ATM:
If PIM address, **remap the address bits**



UM-PIM Motivation: A Uniform Memory Space

- Challenge 1: co-existence of CPU & PIM's pages under a single VM space
- Challenge 2: implementing two different data layouts with low cost

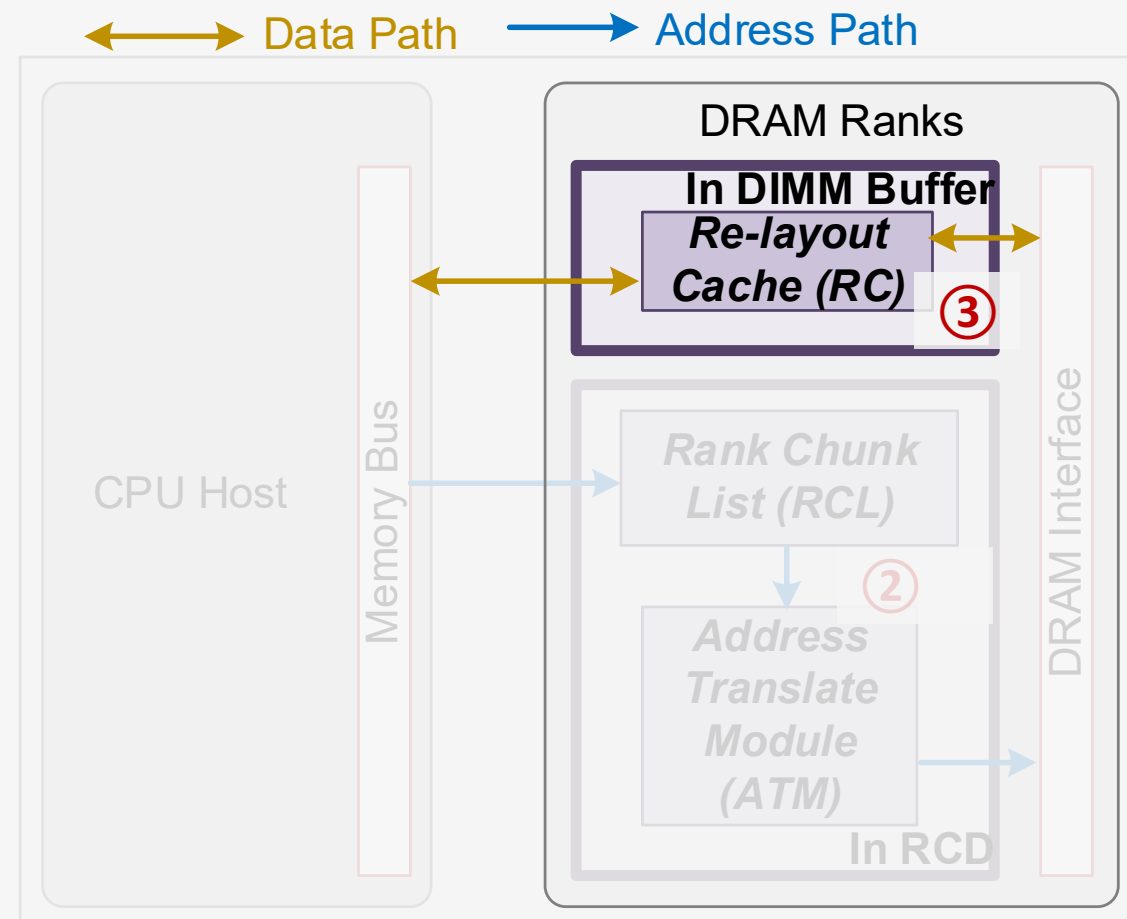




B. Data Layout -- Overview



- ① B.1 Cache line layout inside page
 - Dynamic Address Mapping ②
 - DIMM-level Hardware (ATM&RCL)
- ② B.2 Byte layout inside Cache Line
 - Data re-layout ③
 - PIM-side hardware (RC)

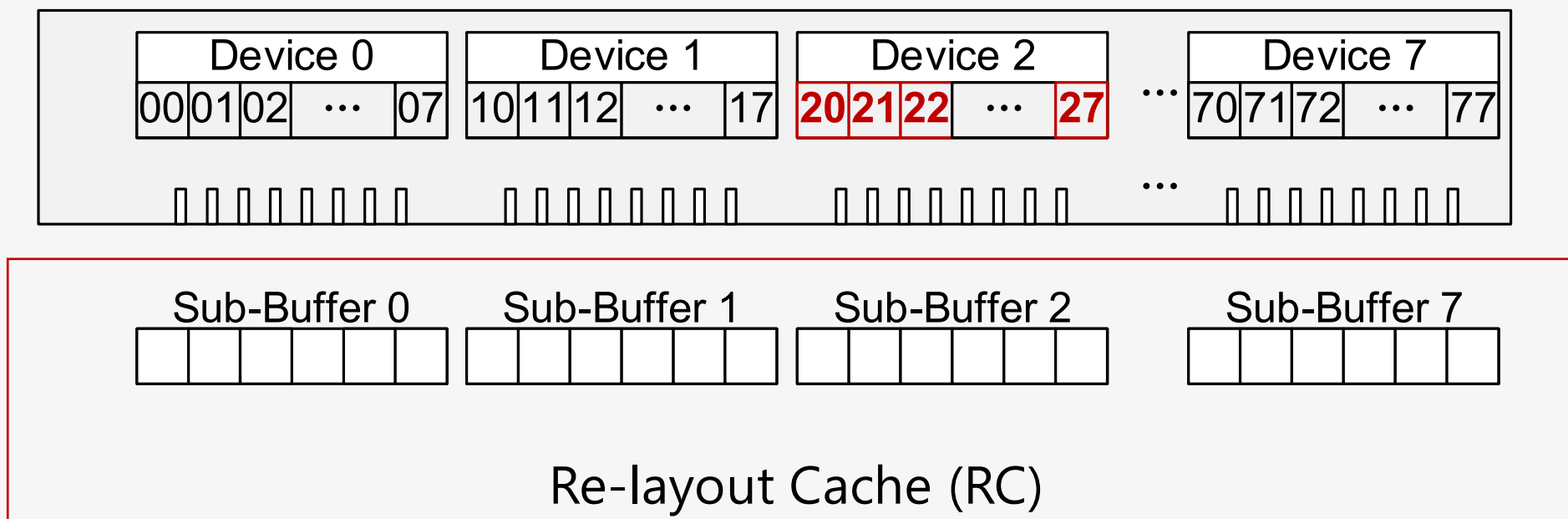




B. Data Layout — Re-layout ③



Ex. of how RC working: Read a 64B cache line of PIM page in Device 2



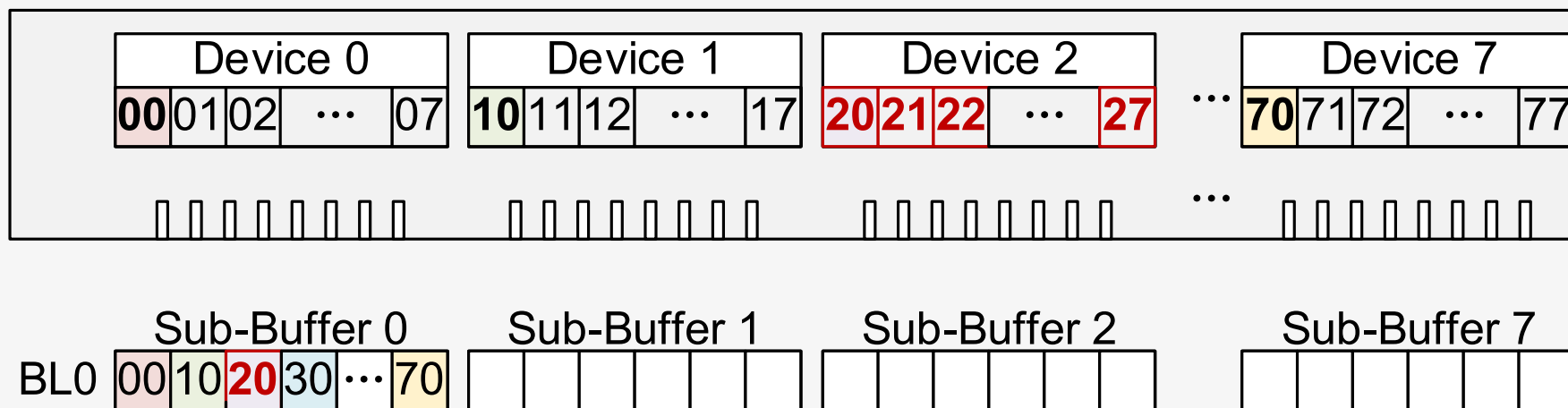


B. Data Layout — Re-layout ③



Ex. of how RC working: Read a 64B Cache Line of PIM Page in Device 2

- **Cycle 0-7:** Read 8 Burst form every device, cache the Bursts in RC



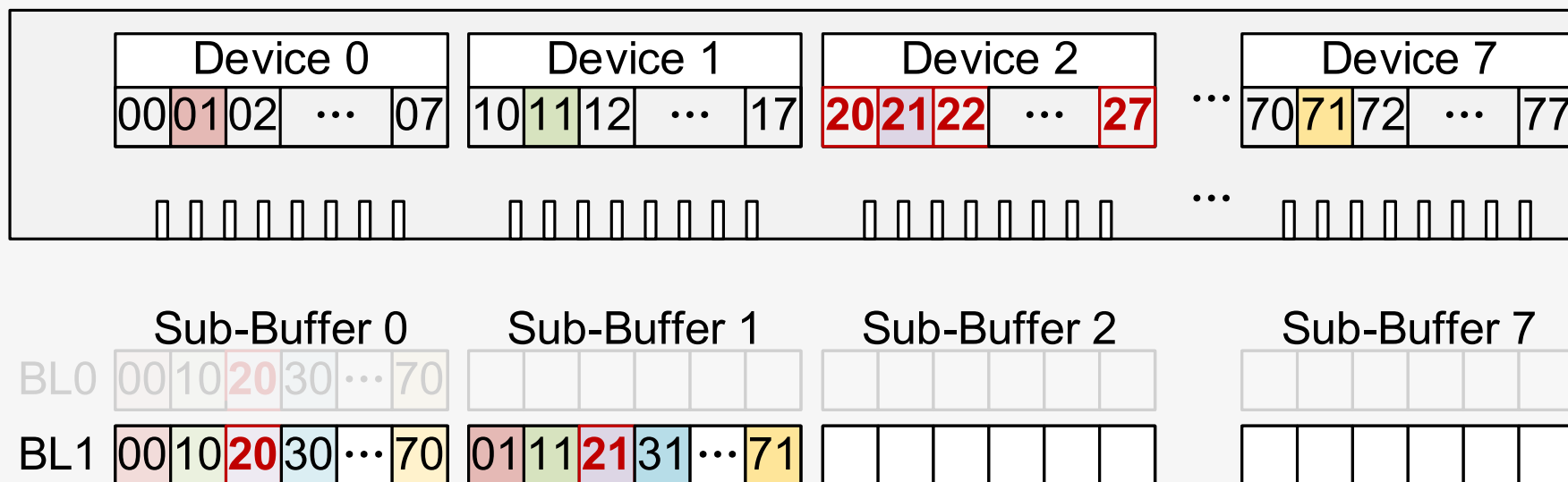


B. Data Layout — Re-layout ③



Ex. of how RC working: Read a 64B Cache Line of PIM Page in Device 2

- **Cycle 0-7:** Read 8 Burst form every device, cache the Bursts in RC



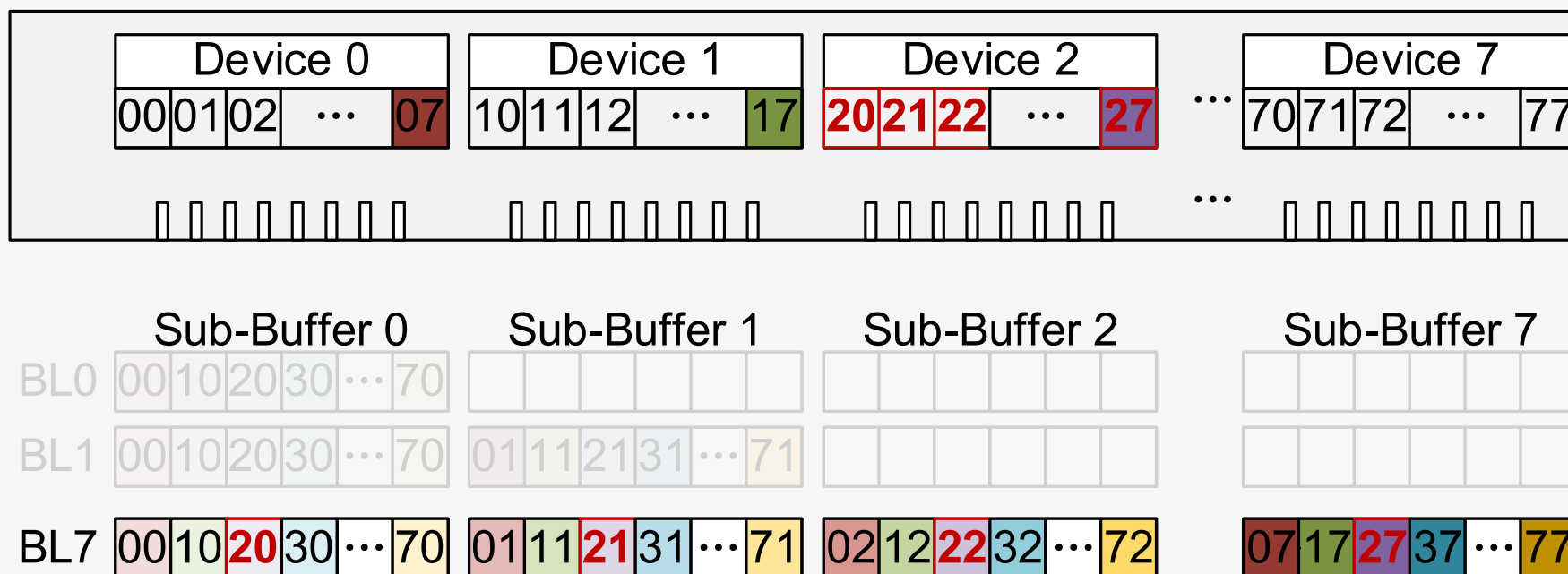


B. Data Layout — Re-layout ③



Ex. of how RC working: Read a 64B Cache Line of PIM Page in Device 2

- **Cycle 0-7:** Read 8 Burst form every device, cache the Bursts in RC



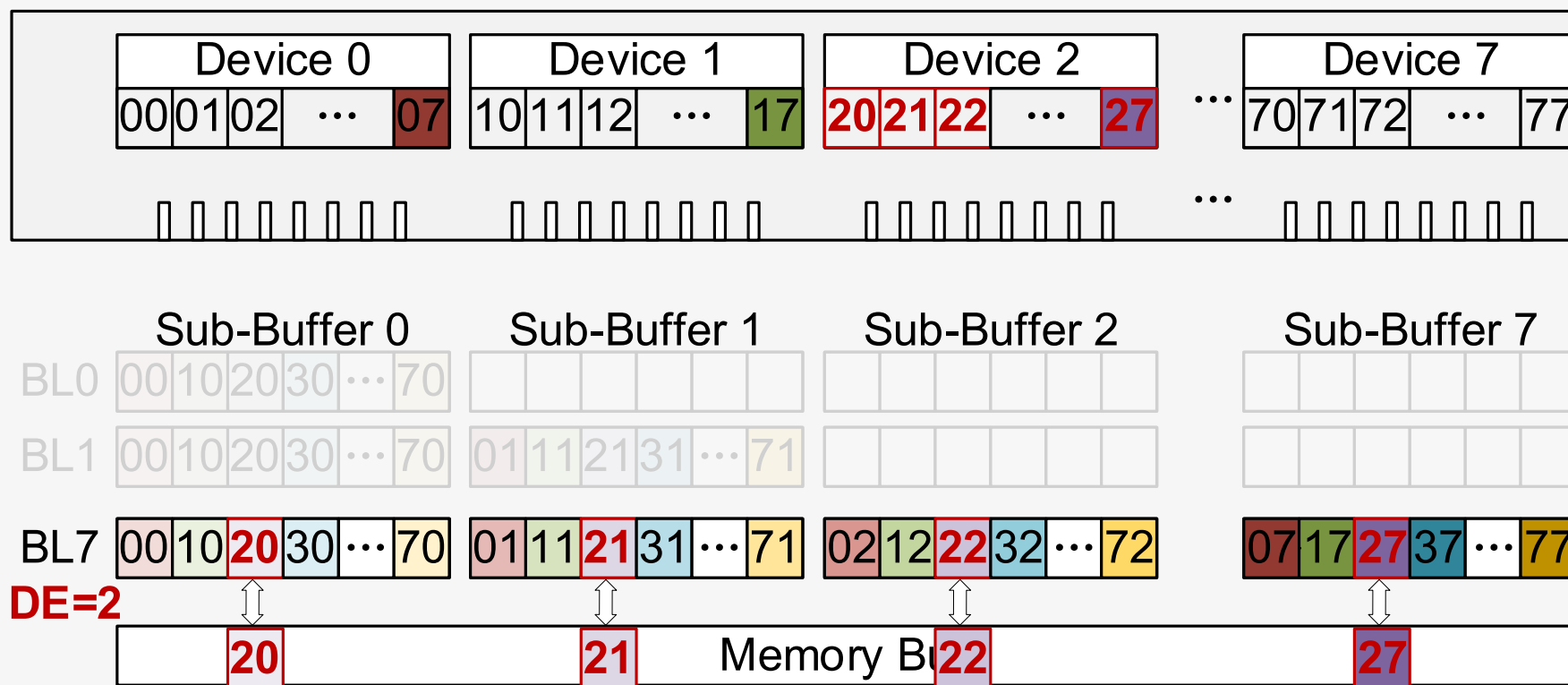


B. Data Layout — Re-layout ③



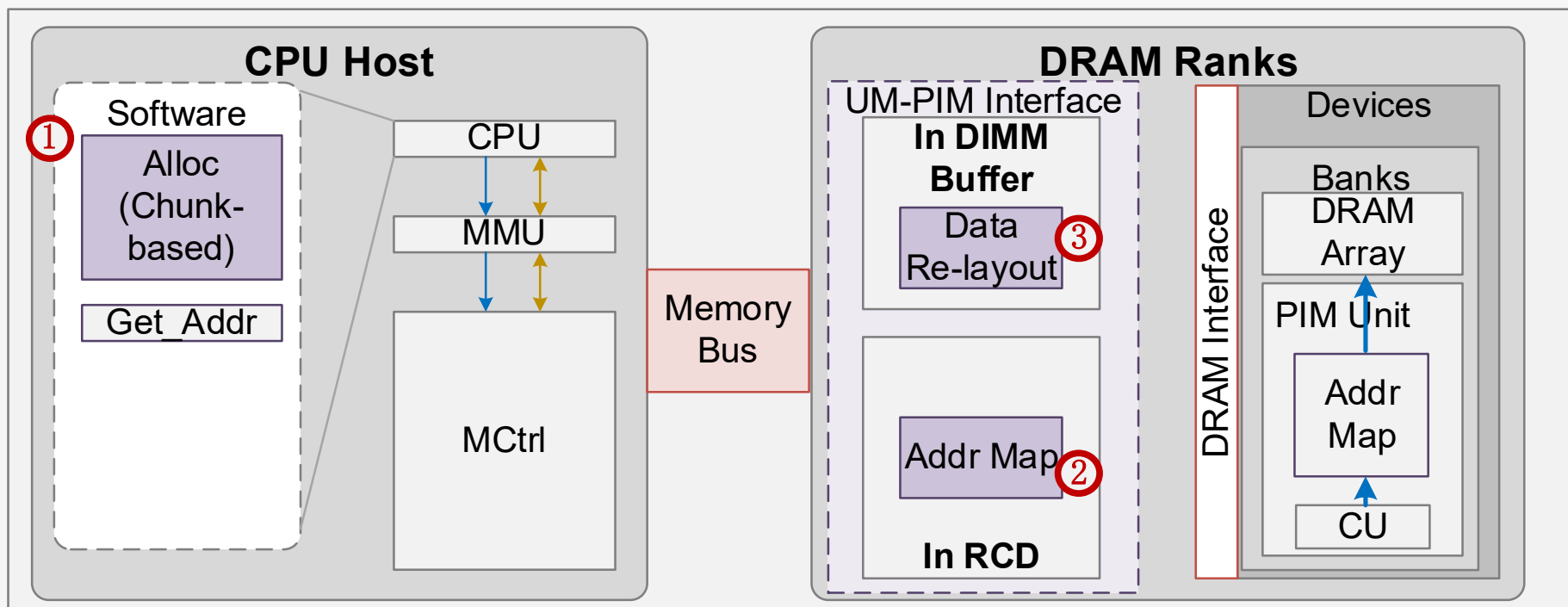
Ex. of how RC working: Read a 64B Cache Line of PIM Page in Device 2

- Cycle 0-7: Read 8 Burst form every device, cache the Bursts in RC
- **Cycle 8:** Get the data from Device 2, form a burst read to CPU





- ① **A.** Memory Management: ① API for CPU Allocating PIM Pages
- ② ③ **B.** Data Layout: ② ③ Hardware Modules on DRAM Side

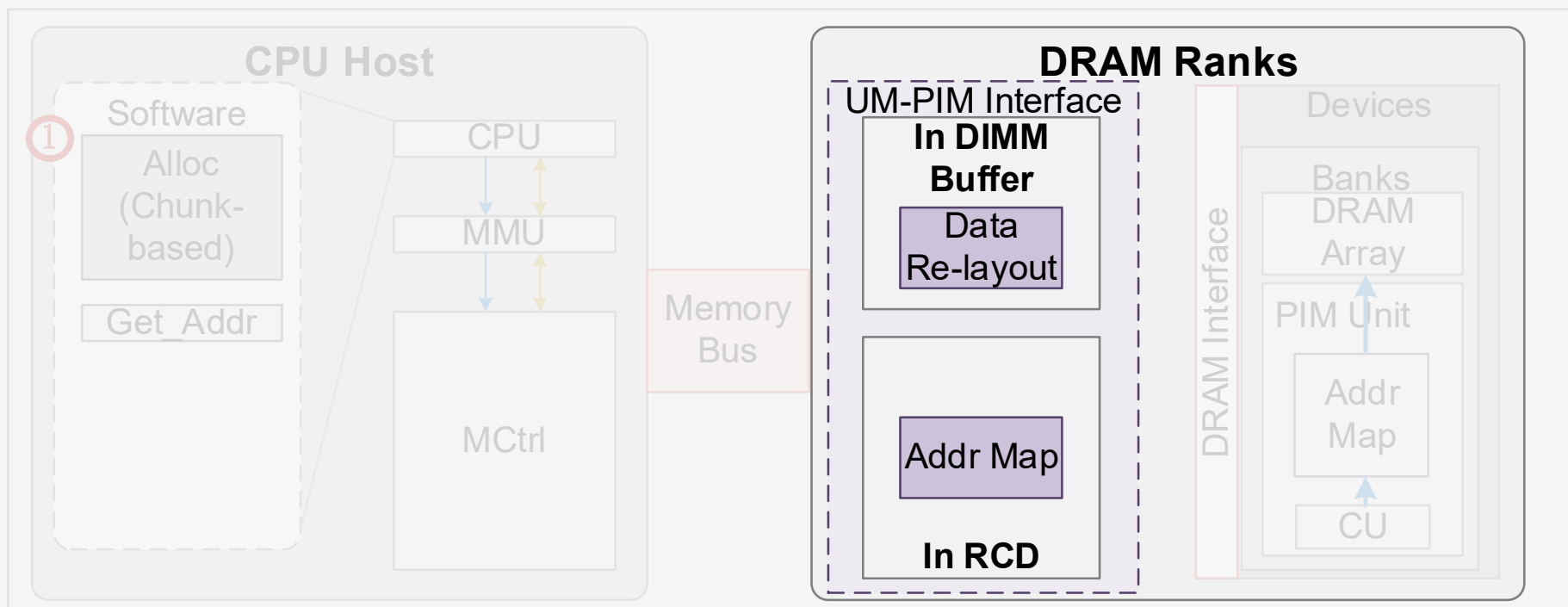




① A. Memory Management: ① API for CPU Allocating PIM Pages

② B. Data Layout: ② ③ Hardware Modules on DRAM Side

**All Hardware modifications are on DRAM side
-- Full compatibility to Soc**





🔴 Accelerating CPU access PIM pages (SW-HW codesign solution)

🔴 Other details:

- Detailed description of CPU and PIM accessing PIM pages
- Extended DRAM instruction support for UM-PIM
- Inter-PIM-units communication APIs

UM-PIM: DRAM-based PIM with Uniform and Shared Memory Space in ISCA'24

Yilong Zhao, Mingyu Gao, Fangxin Liu*, Yiwei Hu, Zongwu Wang, Han Lin, Ji Li, He Xian, Hanlin Dong, Tao Yang, Naifeng Jing, Xiaoyao Liang, Li Jiang*

Shanghai Jiao Tong University, Tsinghua University and Huawei



Simulator & Configuration

- CPU: GEM5 + Ramulator2; PIM units: **UPMEM DPU @ 500MHz**
- DDR4-2400, 8(Channel)x 8(Ranks)

Benchmarks:

- CPU Workloads: from **SPEC CPU 2006**
 - lbm, mcf, sjeng, dealII, xalancbmk, gcc
- PIM Workloads (requires CPU & PIM): from **PRIM**
 - BFS, PR, MLP, UNI, TC, SCAN-RSS/SSA, SEL, HST, NW, WFA, RL

Baseline:

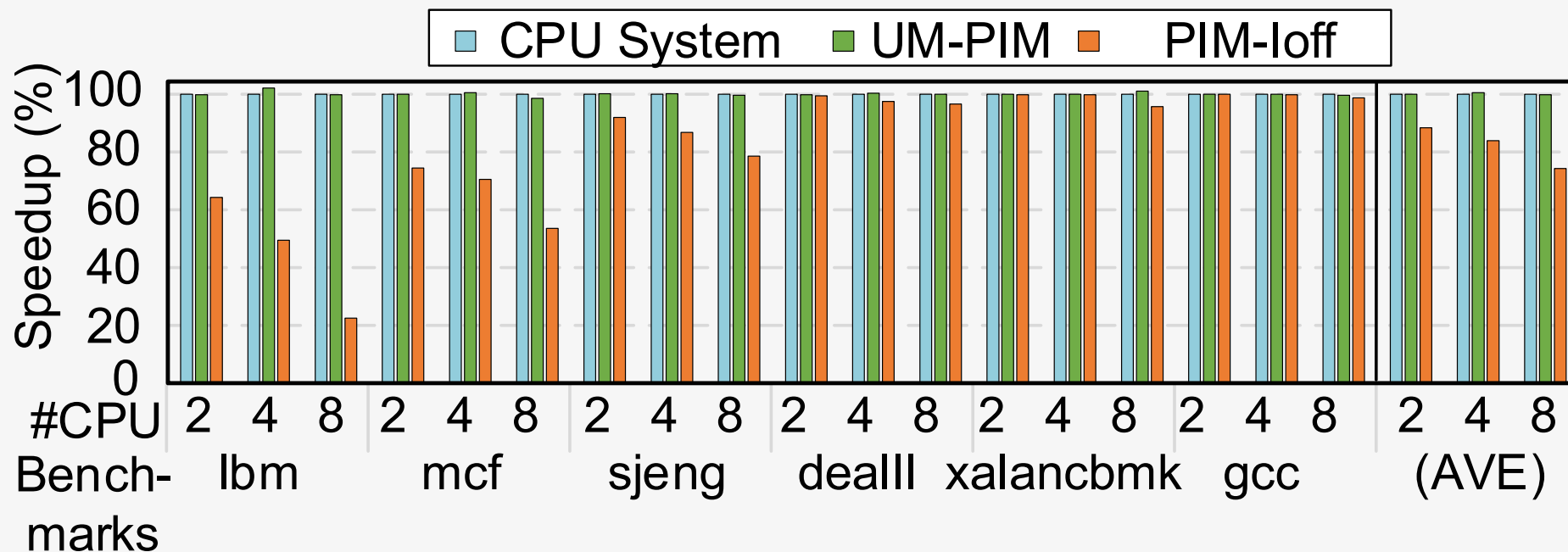
- PIM-Iso: PIM System with **Isolated memory space**
- PIM-Ioff: PIM System with **Rank & Channel interleaving switched off**,



Results on CPU Workloads



UM-PIM does not switch off the interleaving



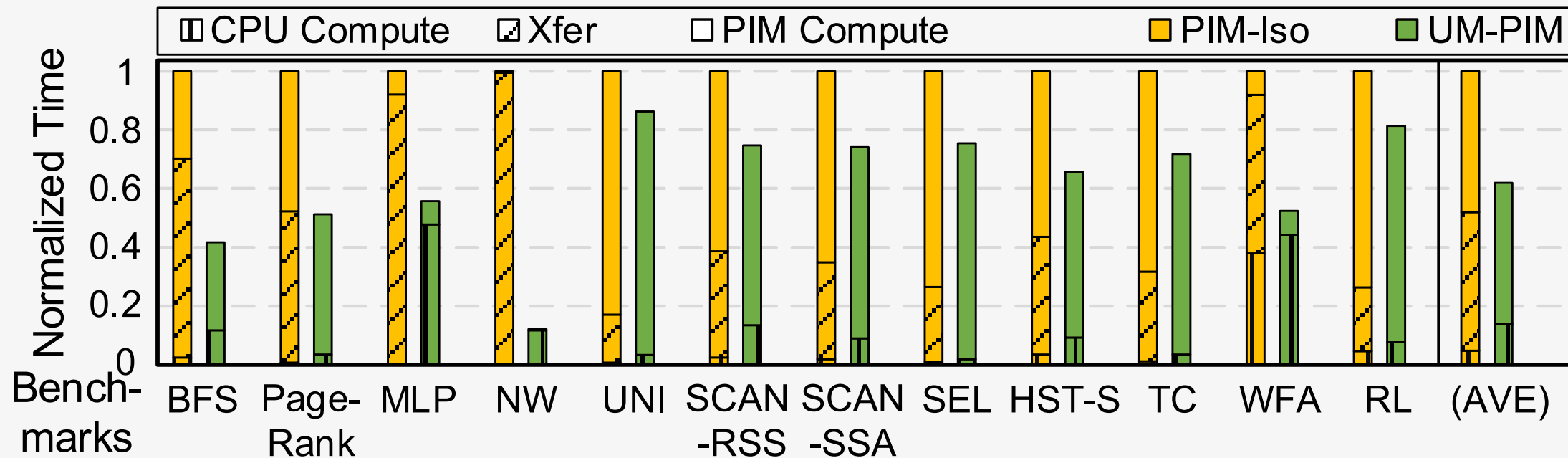
- PIM-Ioff: **22%** performance loss in average, interleaving switched off
- More CPUs in the system, larger perf. loss
- UM-PIM: **<0.1%** performance loss



Results on PIM Workloads



UM-PIM has no data transfer between isolated memory spaces



- Compared to PIM-Iso, UM-PIM has **4.93×** reduction on CPU time (including Computing and Data Transfer), resulting in **1.96×** end-to-end speedup in average.



Key advantages of UM-PIM:

- **Uniform & shared memory space:** CPU and PIM pages co-exist in a uniform memory space, and data transfer is eliminated compared to isolated space design
- **Fast & Transparent data layout:** Dynamic address mapping and data re-layout for two kinds of pages are processed fast and transparently by DRAM-side hardware module
- **Compatibility with current CPU systems:** no modifications on CPU-side hardware and CPU page's memory management.



[ISCA'24] UM-PIM: DRAM-based PIM with Uniform and Shared Memory Space

Yilong Zhao, Mingyu Gao, Fangxin Liu*, Yiwei Hu, Zongwu Wang, Han Lin, Ji Li, He Xian, Hanlin Dong, Tao Yang, Naifeng Jing, Xiaoyao Liang, Li Jiang*

Shanghai Jiao Tong University, Tsinghua University and Huawei

[ASPLOS'25] PUSHtap: PIM-based In-Memory HTAP with Unified Data Storage Format

Yilong Zhao, Mingyu Gao, Huanchen Zhang, Fangxin Liu,* Gongye Chen, He Xian, Haibing Guan, and Li Jiang*

Shanghai Jiao Tong University, Tsinghua University

[ISCA'26] COSM: A Cooperative Scheduling Framework for Concurrent PIM and CPU Execution on Mobile Devices

Yilong Zhao=, Fangxin Liu=, Onur Mutlu, Mingyu Gao, Jian Liu, Haibing Guan, and Li Jiang*

Shanghai Jiao Tong University, ETH Zurich, Tsinghua University and Beihang University

Monday, Jun 29 10:15 – 11:55 AM
Ballroom A, Session 1A



- **Background: HTAP Database**
2 workloads: **OLTP** (row-wise transactions) and **OLAP** (column-wise analytics).
- **Problem** : **Row-store vs. column-store** trade-off -- no format fits both
- **Insight**: Exploit access divergence -- **CPU interleaved & PIM contiguous** access
→ 1 data format serving 2 workloads: **PIM-enabled unified storage for HTAP**.
- **Key Challenge** → **Solutions** :

Alignment of Heterogeneous
Data Types



Unified Data Format

- **Evaluation** : Zsim+Ramulator2 simulation | 8-Channel DDR5 | 20GB CH Benchmark
- **Results** : 3.4× peak OLTP throughput | 4.4× OLAP throughput under peak OLTP (vs. MI baseline)



HTAP: The Convergence of Conflicting Workloads



Hybrid transaction/analytical processing (HTAP) database, 2 processing sets:

Online transaction processing (OLTP):

Access Pattern:

Row-wise, random

Bottleneck:

Computation-intensive

Ideal Format:

Row-store

Online analytical processing (OLAP):

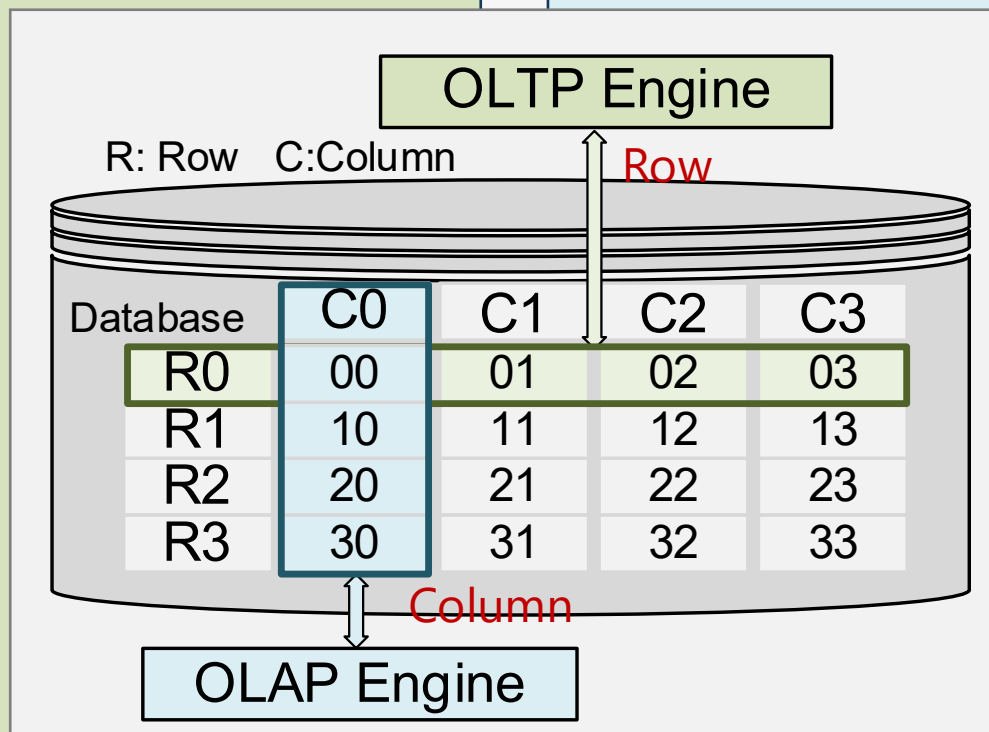
Access Pattern:

Column-wise, sequential

Bottleneck:

Memory-intensive

Ideal Format:
Column-store





HTAP: The Convergence of Conflicting Workloads



Hybrid transaction/analytical processing (HTAP) database, 2 processing sets:

Online transaction processing (OLTP):

Access Pattern:

Row-wise, random

Bottleneck:

Computation-intensive

Ideal Format:
Row-store

Online analytical processing (OLAP):

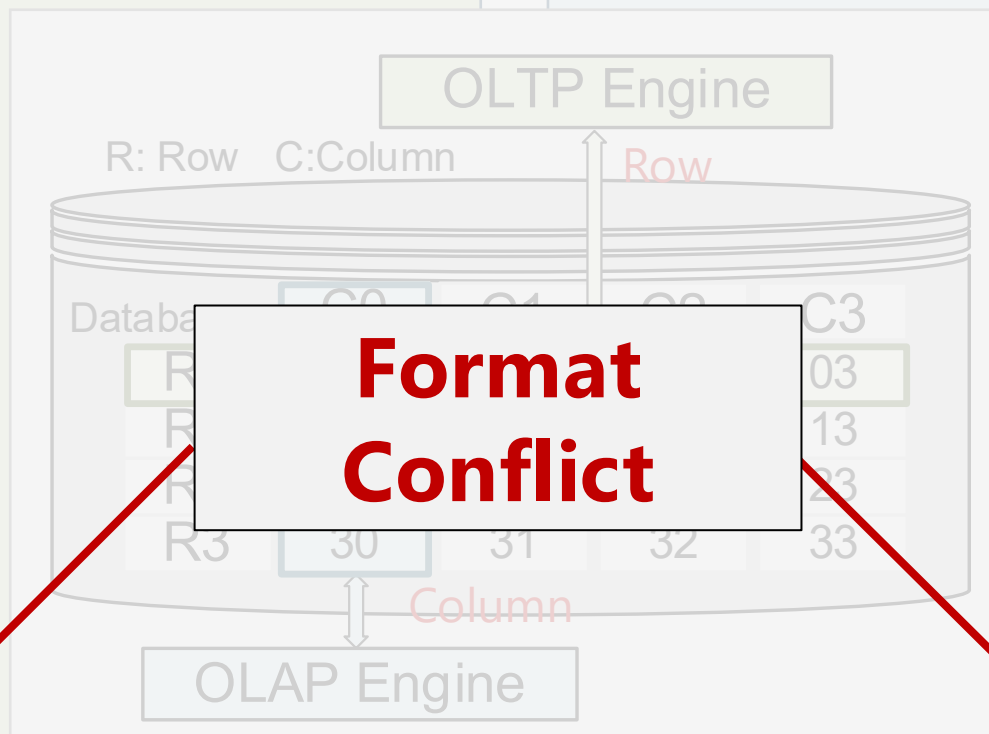
Access Pattern:

Column-wise, sequential

Bottleneck:

Memory-intensive

Ideal Format:
Column-store





Access Divergence in CPU-PIM System

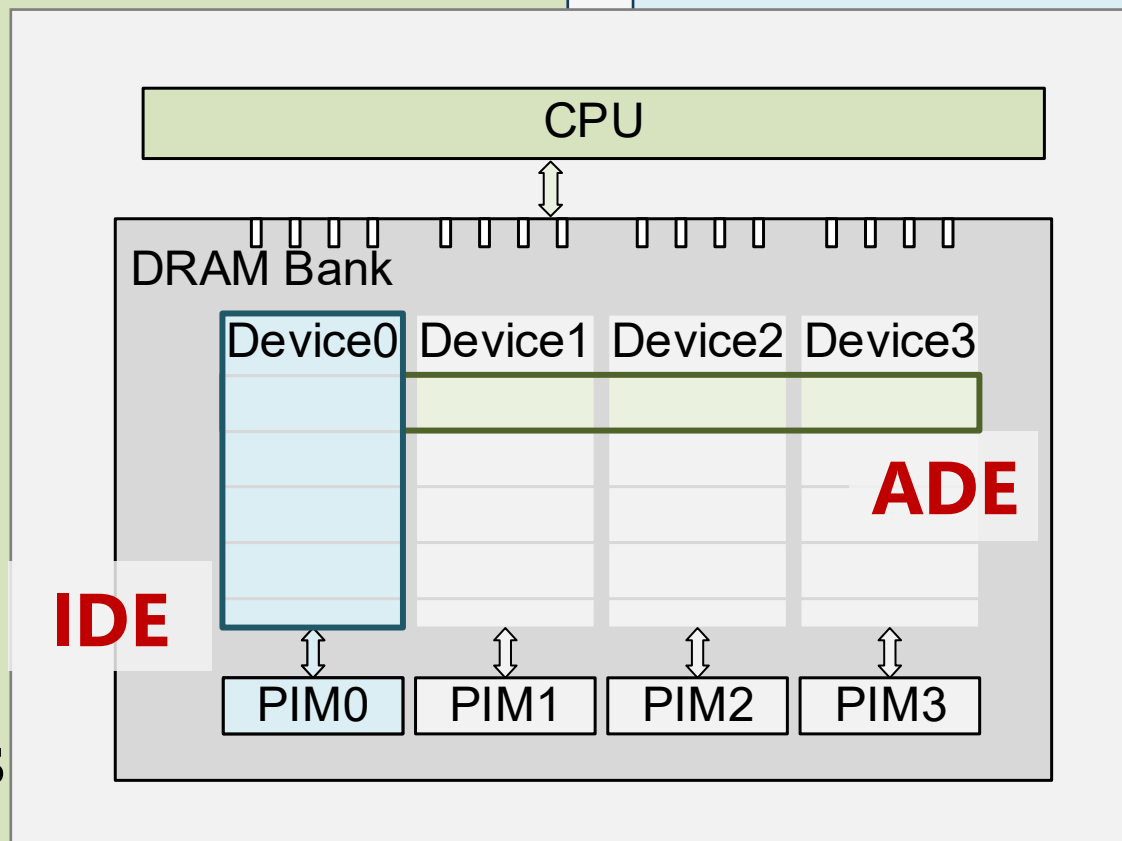


CPU

PIM

Interleaved Access
for high bandwidth

→ Across devices
(**ADE**) Access

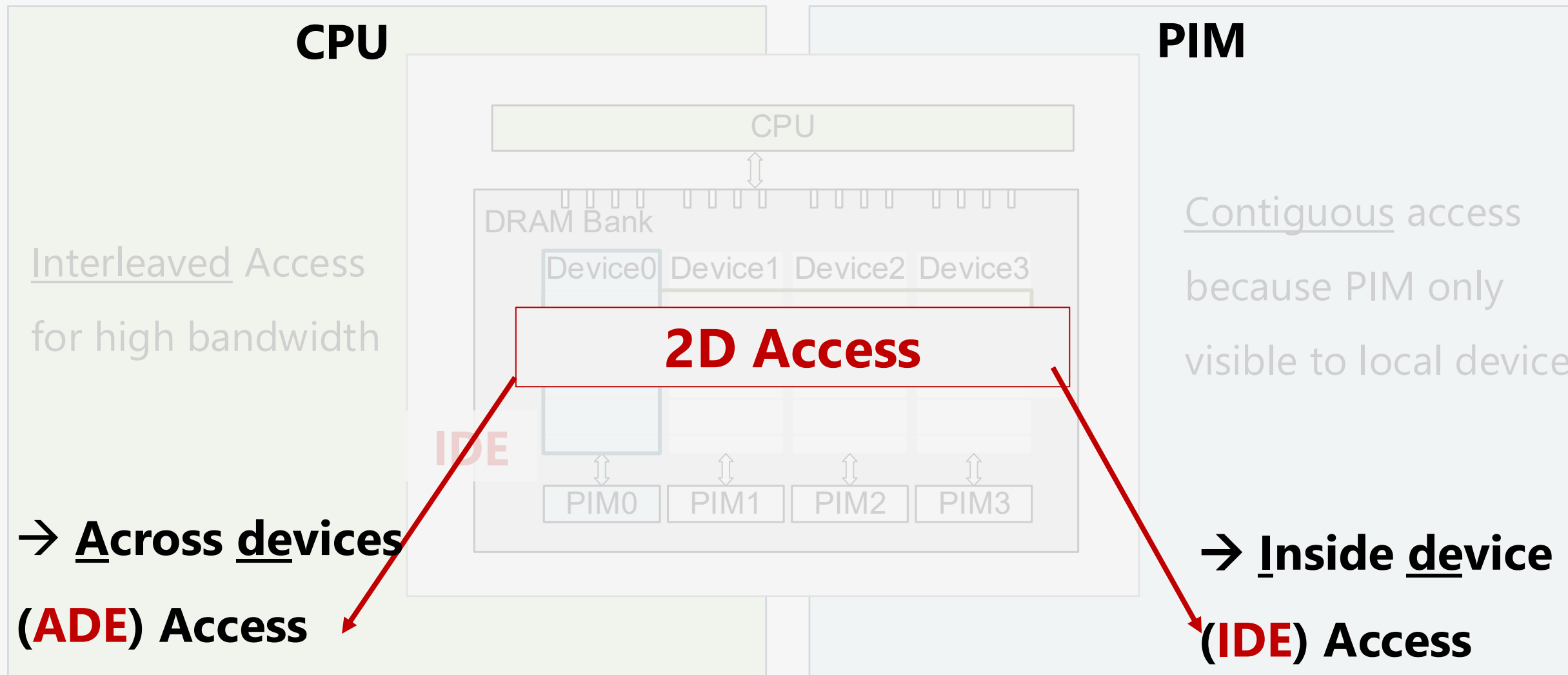


Contiguous access
because PIM only
visible to local device

→ Inside device
(**IDE**) Access



Access Divergence in CPU-PIM System



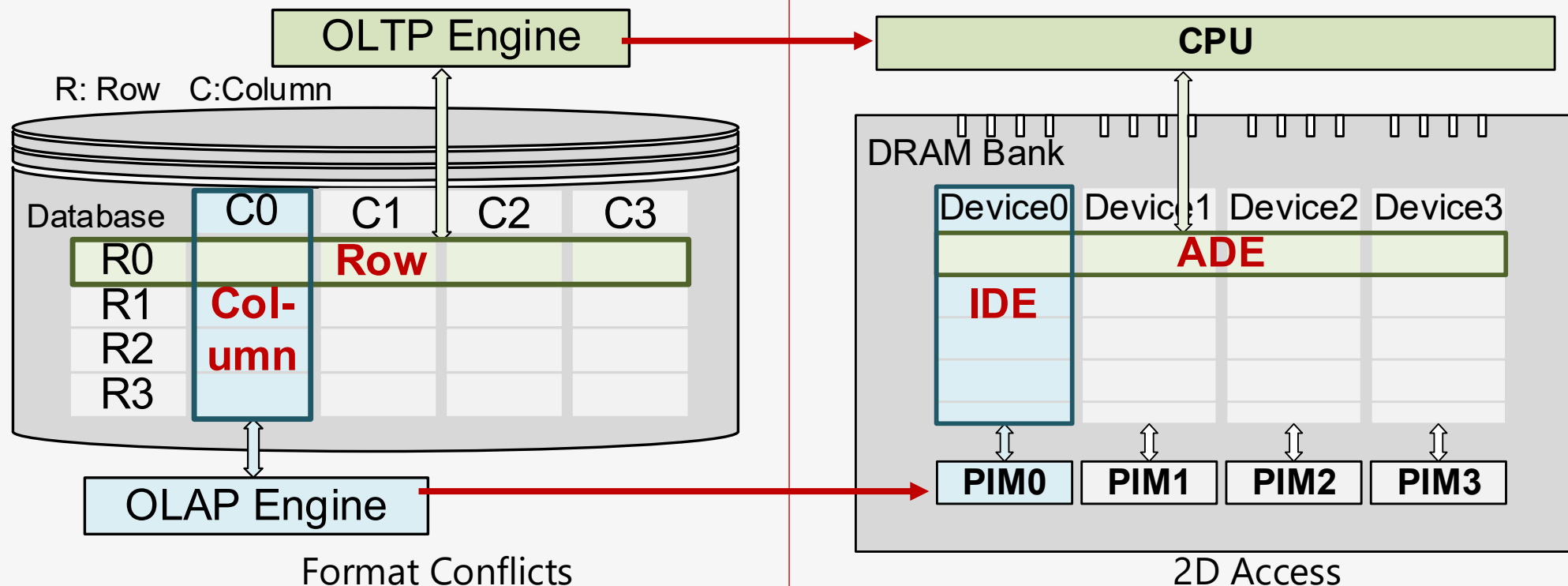


Key Insight: Format Conflict vs. Access Divergence

HTAP

OLTP	Row-wise	→	Across Device (ADE)	CPU
OLAP	Column-wise	→	Inside Device (IDE)	PIM

PIM





The PUSHtap Solution: Hardware-Software Co-alignment

Objective: Single Instance, Unified Format

-- Satisfies both row & column operations, simultaneously achieve 3 goals

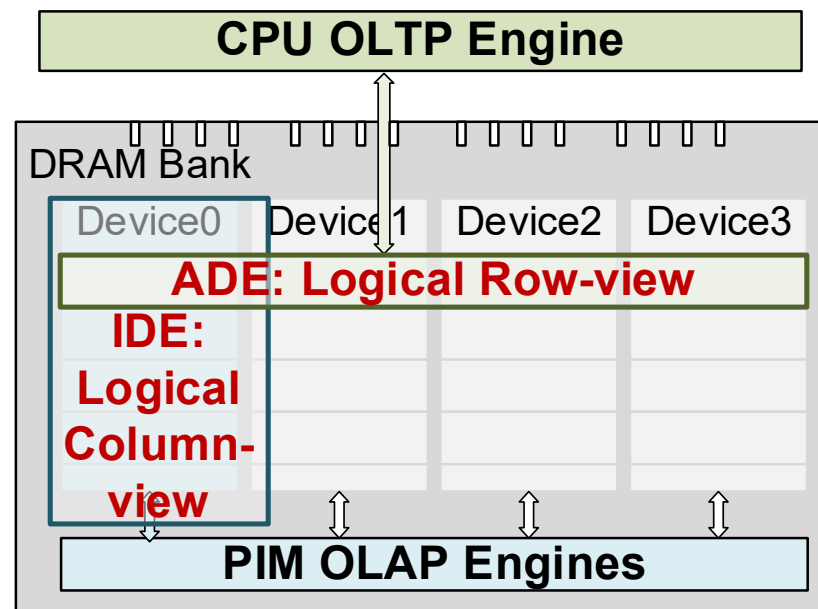
- Workload-specific Optimization:
- Performance Isolation:
- Data Freshness:

*Unified format -> both row & column access
No consistency (merge/rebuild) overhead
Processed on the same instance*

Core Approach: Unified Data Format

Logical Rows: **Aligned** to ADE

Logical Columns: **Aligned** to IDE





Challenge 1: Data Format Challenge (1)

Our goal requires to align data in the IDE & ADE direction

Format challenge:

Software:

Diverse column widths

2~9 Bytes

id 2B	d_id 2B	w_id 4B	zip 9B

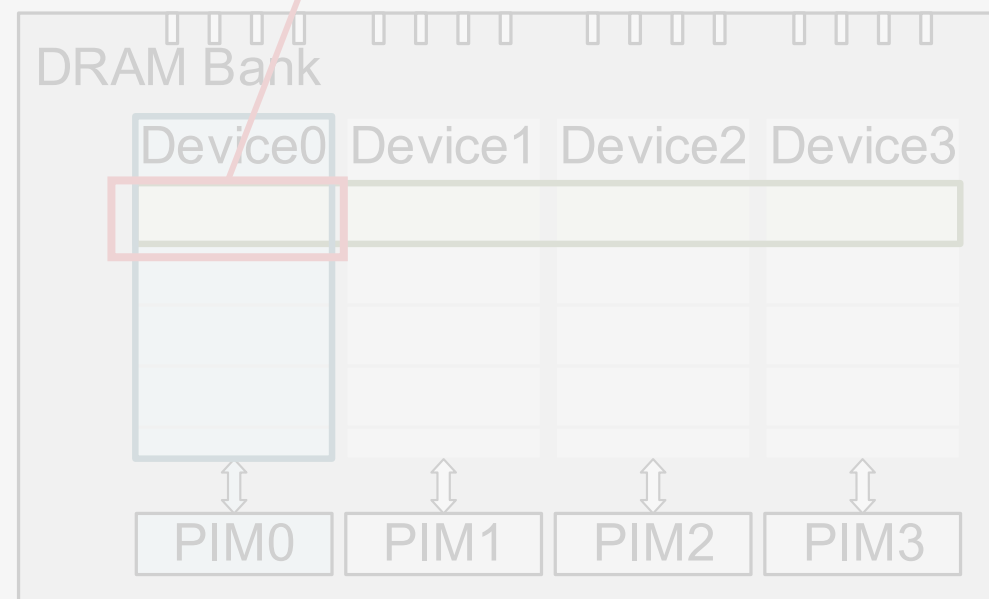
An Example of Table

vs.

Hardware:

Fixed interleaving granularity

Typically 4 Bytes



A DRAM Rank



Challenge 1: Data Format Challenge (1)

Our goal requires to align data in the IDE & ADE direction

Format challenge:

Software:

Diverse column widths

2~9 Bytes

id 2B	d_id 2B	w_id 4B	zip 9B

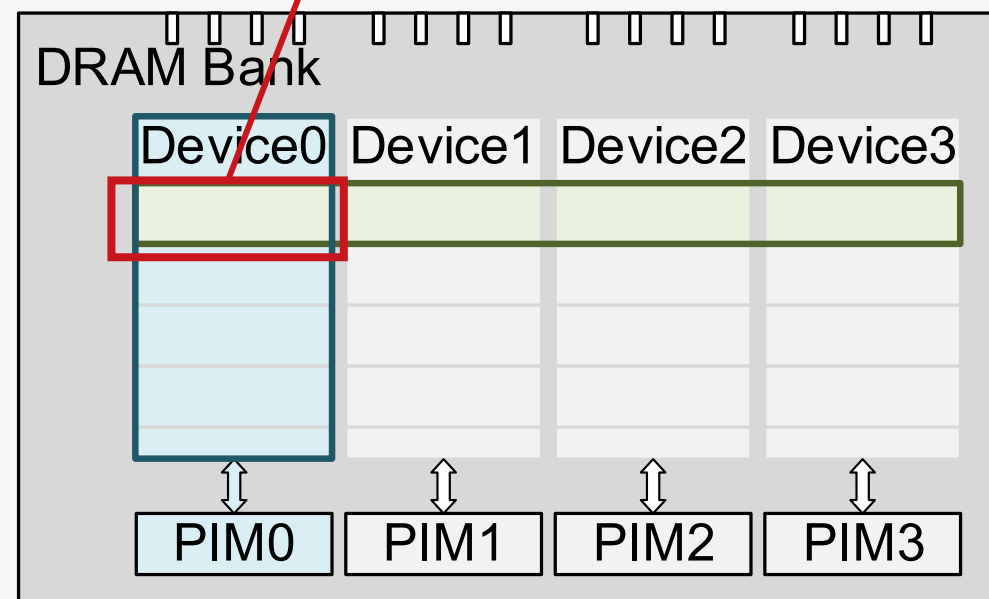
An Example of Table

vs.

Hardware:

Fixed interleaving granularity

Typically 4 Bytes



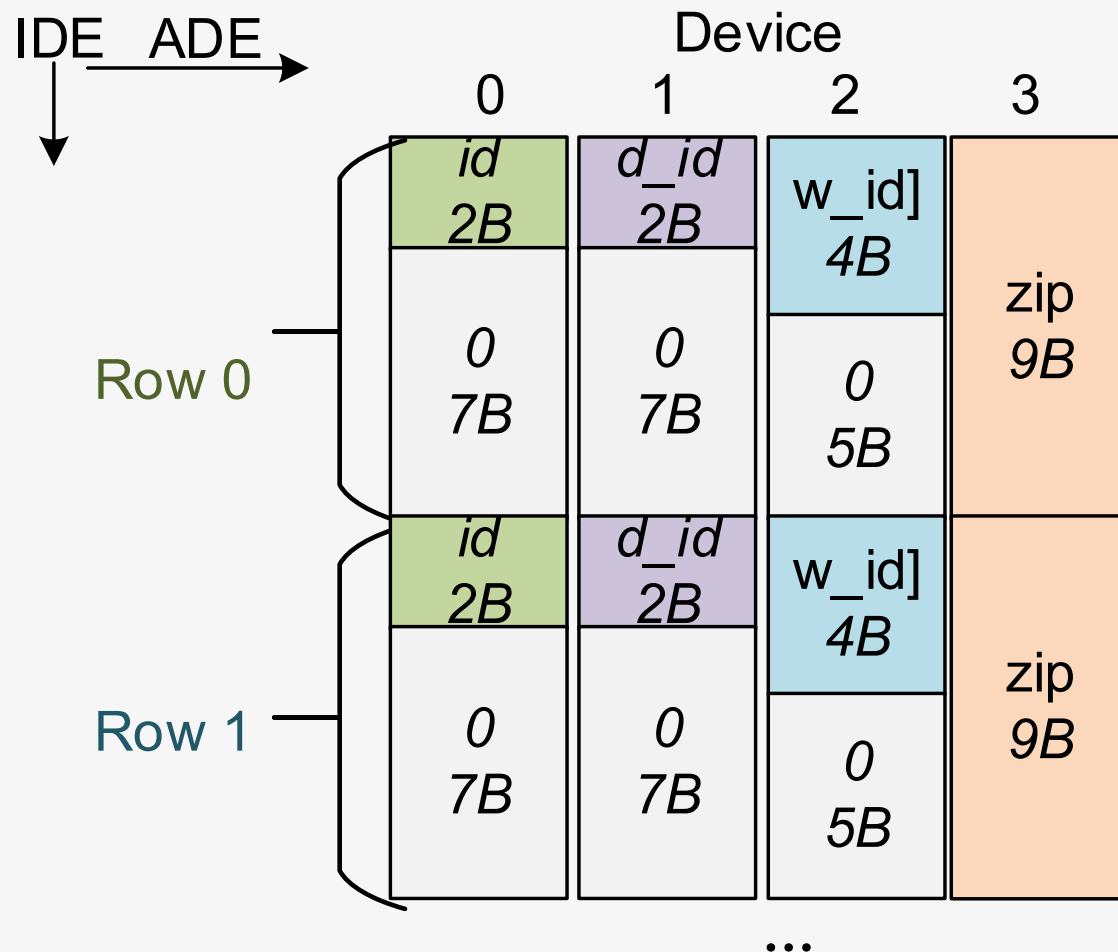
A DRAM Rank



Challenge 1: Data Format Challenge (2)



One Naïve Solution: padding 0s



Aligned on both dimensions ✓

Too many 0s (51.2%) ✗

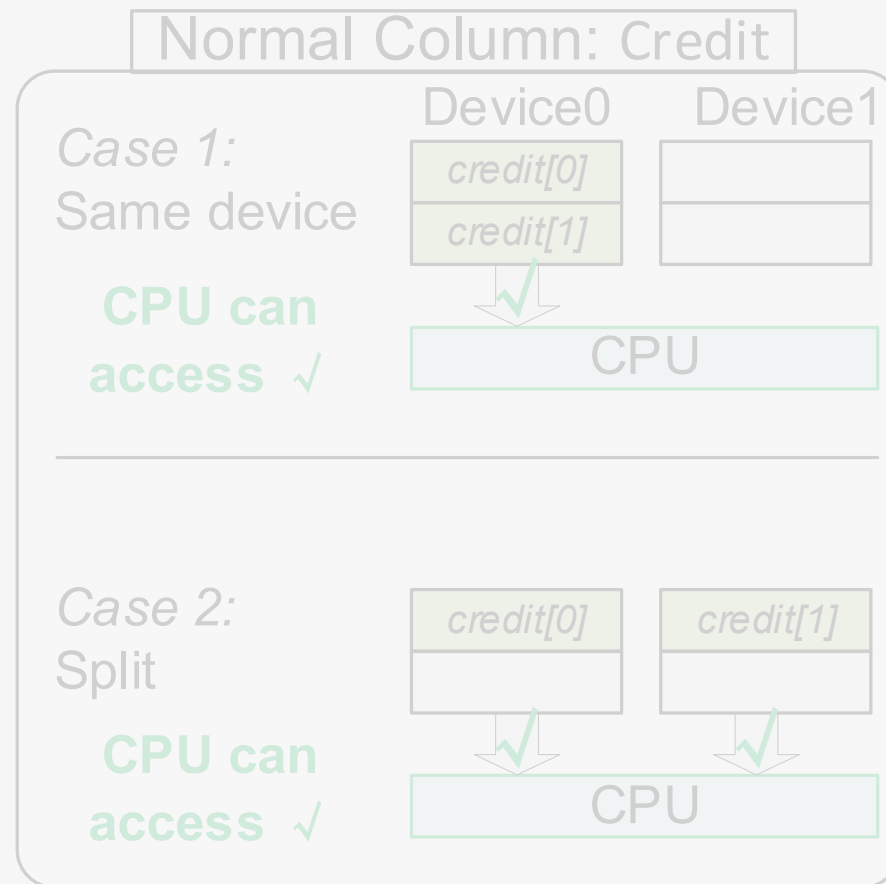
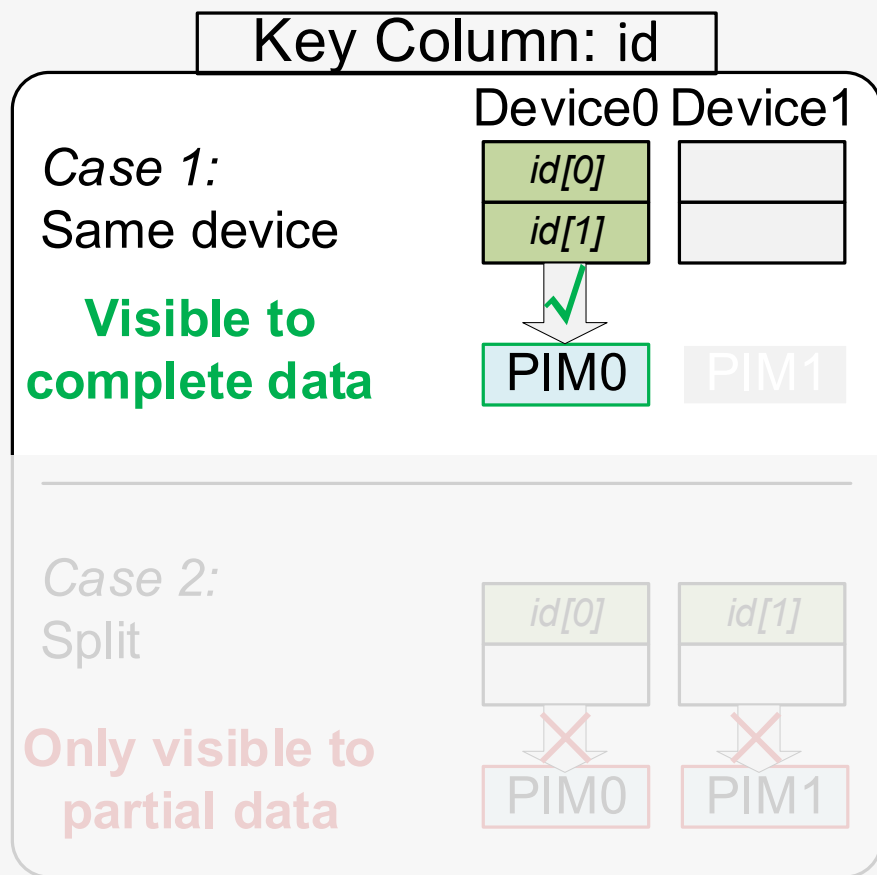
How to reduce the 0s?



Solution 1: Compact Aligned Format (1)

Row Categorization:

- **Key columns** : Frequently scanned by OLAP → Should be on the same device
- **Normal columns** : Never scanned by OLAP → Can be split to multiple devices

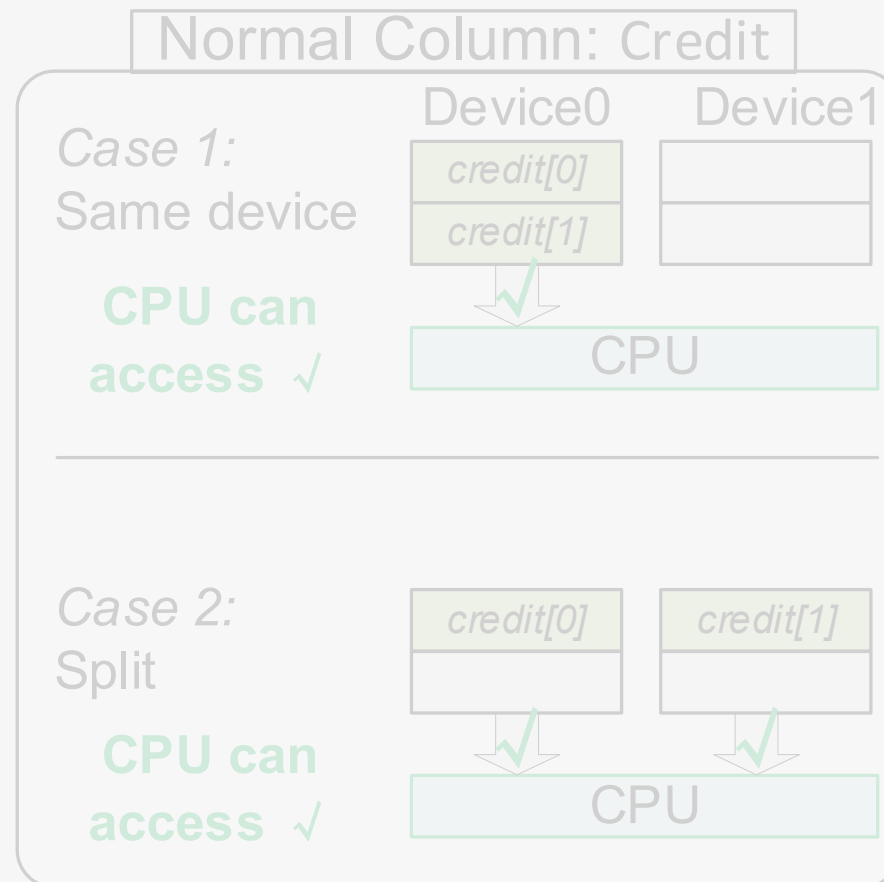
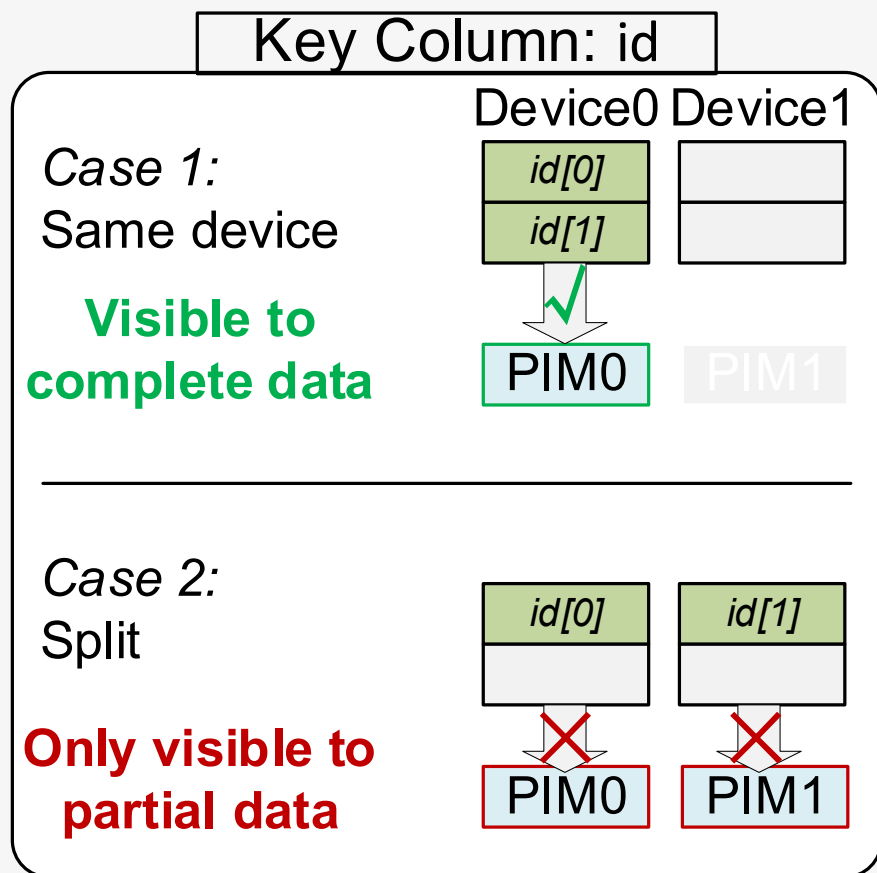




Solution 1: Compact Aligned Format (1)

Row Categorization:

- **Key columns** : Frequently scanned by OLAP → Should be on the same device
- **Normal columns** : Never scanned by OLAP → Can be split to multiple devices



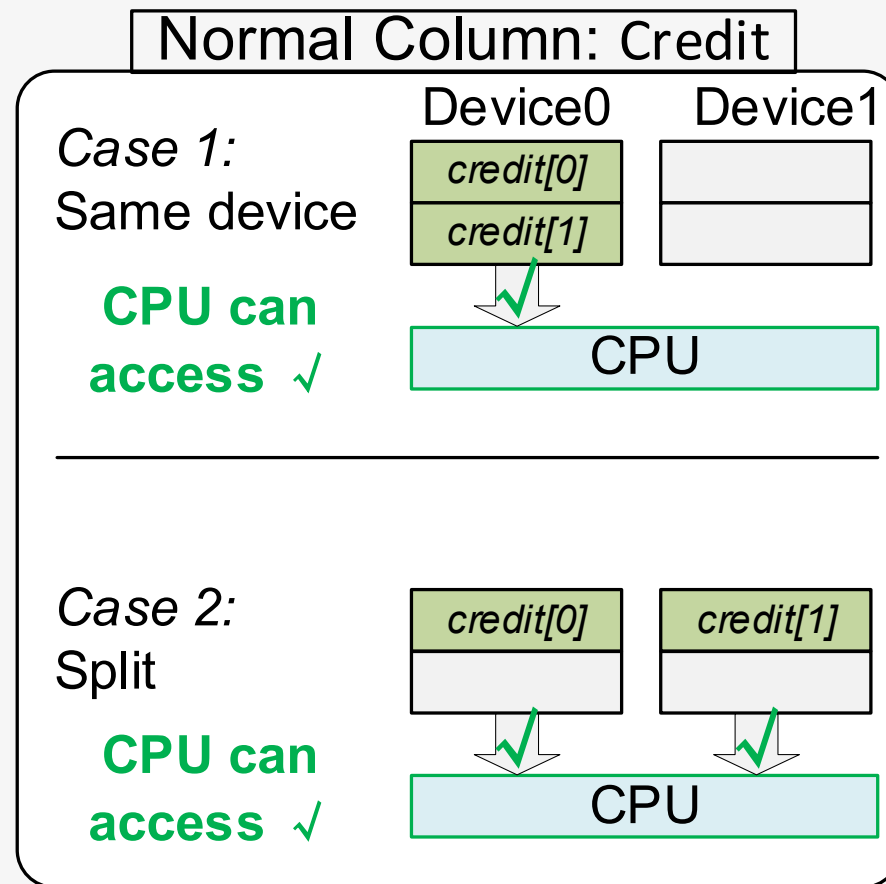
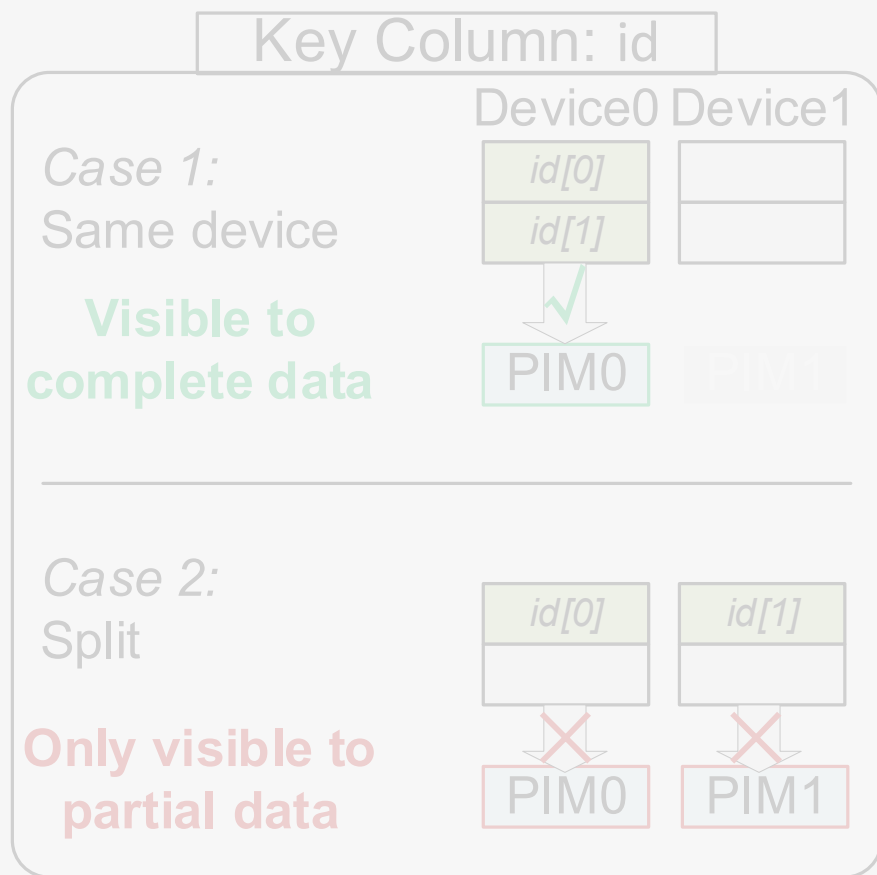


Solution 1: Compact Aligned Format (1)



Row Categorization:

- **Key columns** : Frequently scanned by OLAP → Should be on the same device
- **Normal columns** : Never scanned by OLAP → Can be split to multiple devices

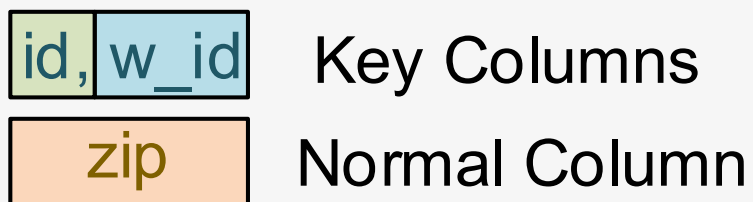
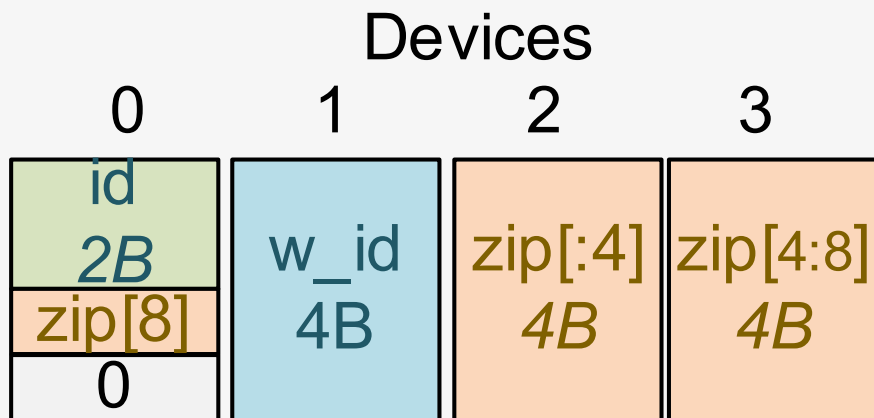




Solution 1: Compact Aligned Format (2)



Compact Aligned Format:



Use *normal columns* (e.g., **zip**) for padding instead of "0"

→ **Fewer "0"s** (6.25%)



- ① PIM Parallelism Optimization
- ① Format-aware Version Control
- ① Architecture Support for Concurrent CPU&PIM Access

[ASPLOS'25] PUSHtap: PIM-based In-Memory HTAP with Unified Data Storage Format

Yilong Zhao, Mingyu Gao, Huanchen Zhang, Fangxin Liu,* Gongye Chen, He Xian, Haibing Guan, and Li Jiang*
Shanghai Jiao Tong University, Tsinghua University



HTAP Benchmark:

- CH-benchmark (TPC-C+TPC-H), implemented based on DBx1000
- size: 20GB

Baselines:

- **RS**: Row-store
- **CS**: Column-store
- **MI**: Multi-instance, row-store on CPU side, column-store on PIM side [ICDE'22], ([ICDE'22] is based on HBM, we fairly adapted it to DDR5)

Simulation:

- CPU model: zsim
- DRAM model: Ramulator2

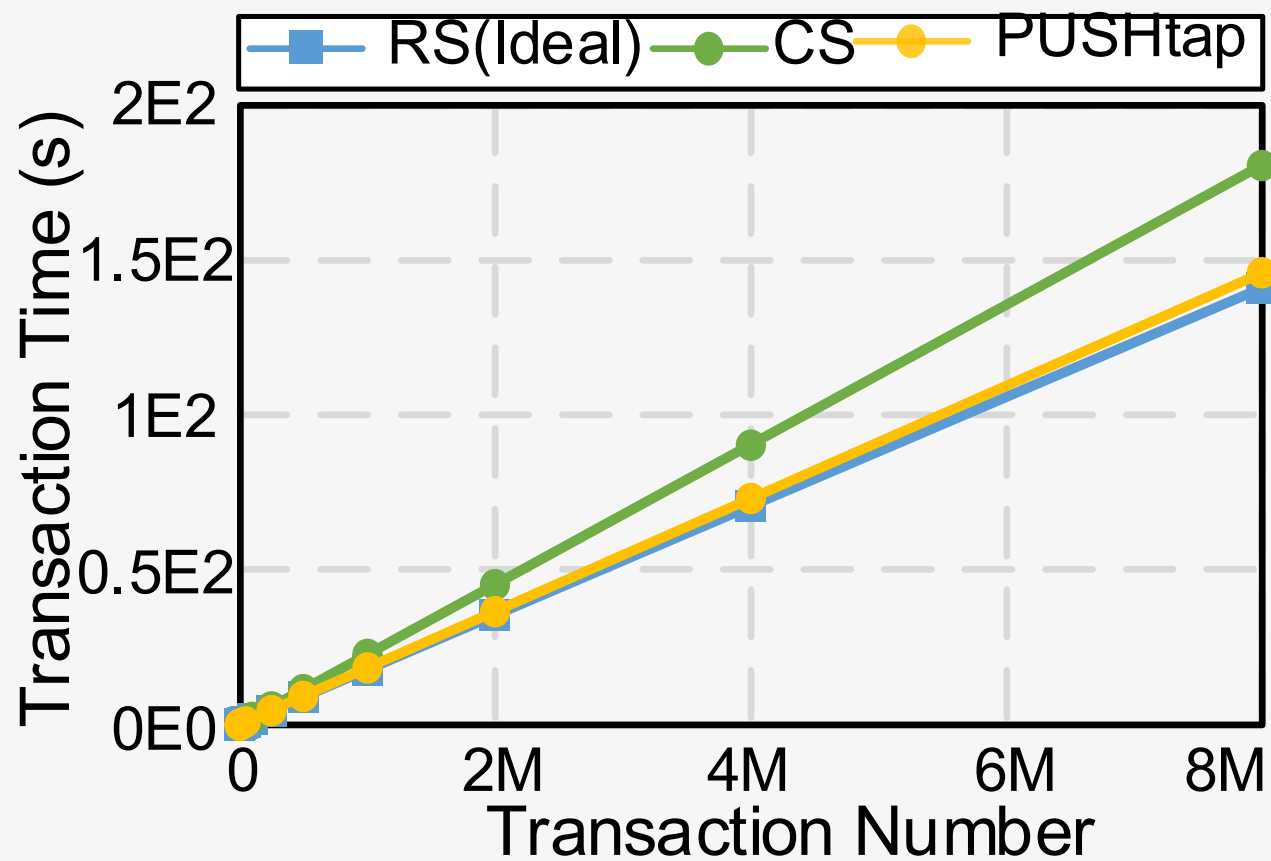
System Configuration	
CPU	16@3.2Hz
DRAM	DDR5-3200, {4 PIM Channel, 4 CPU Channel}*4 Ranks
PIM	500MHz, 64 per Rank



Result: Standalone Performance -- OLTP



OLTP Performance:

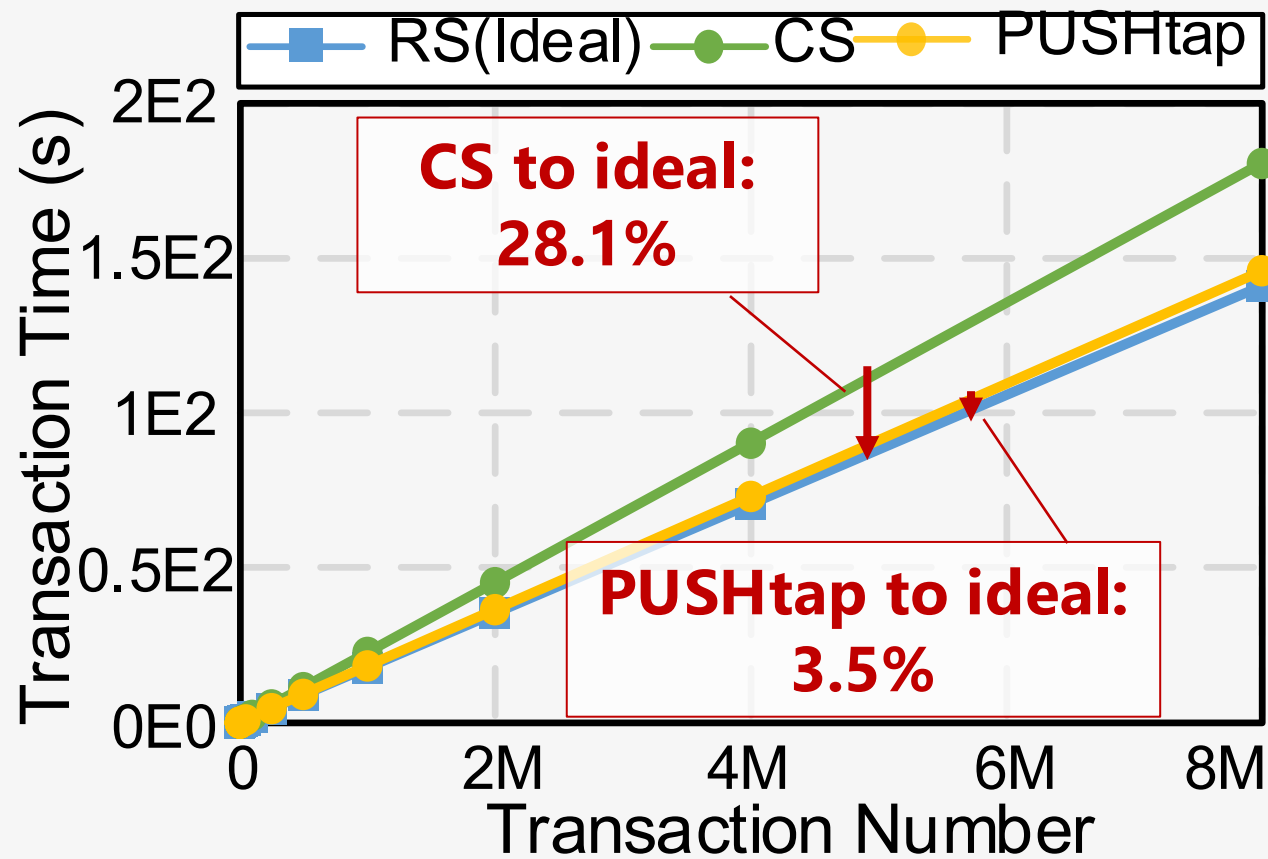




Result: Standalone Performance -- OLTP



OLTP Performance:



- Compared to *Row-store(RS, ideal)*:
- *Column-store (CS)* is 28.1% worse than *ideal*
- ***PUSHtap* only incurs 3.5% degradation**



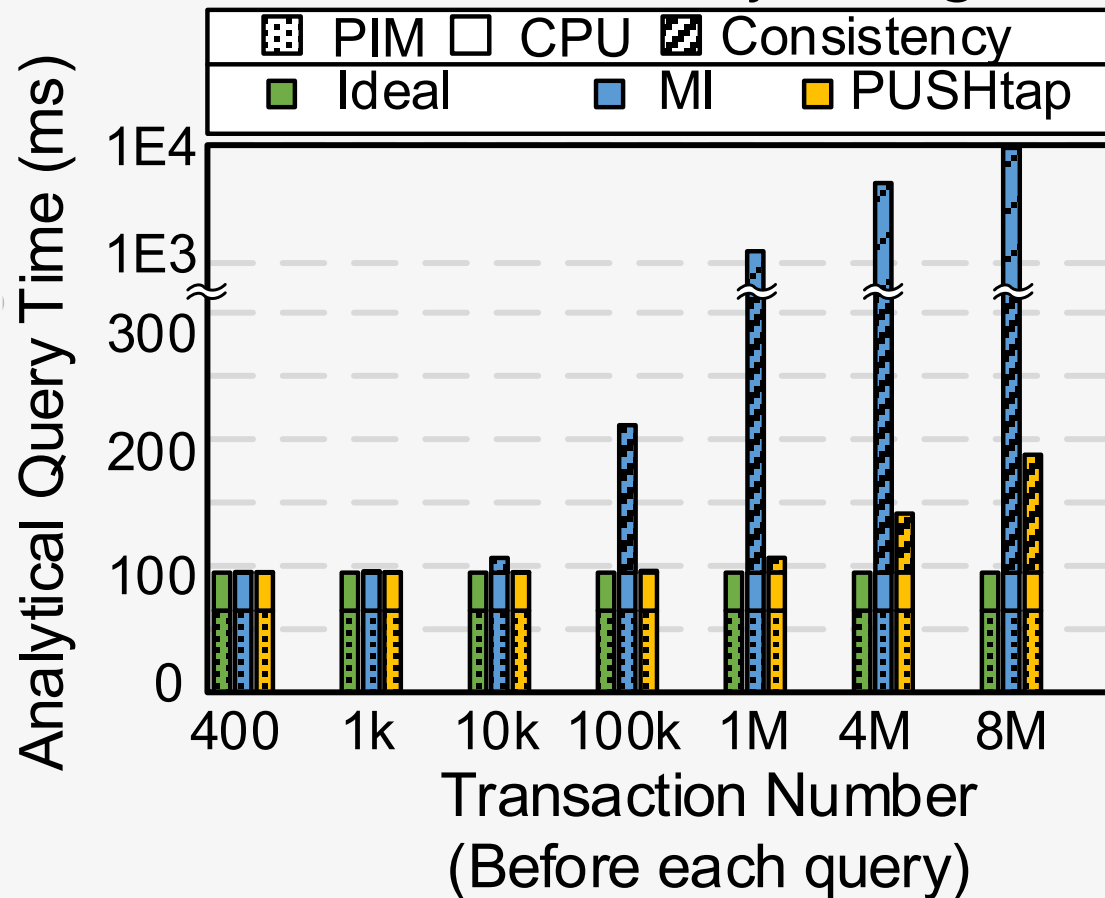
Result: Standalone Performance -- OLAP



OLAP Performance:

ideal: with zero-latency defragmentation

Consistency includes
defragmentation/
rebuilding





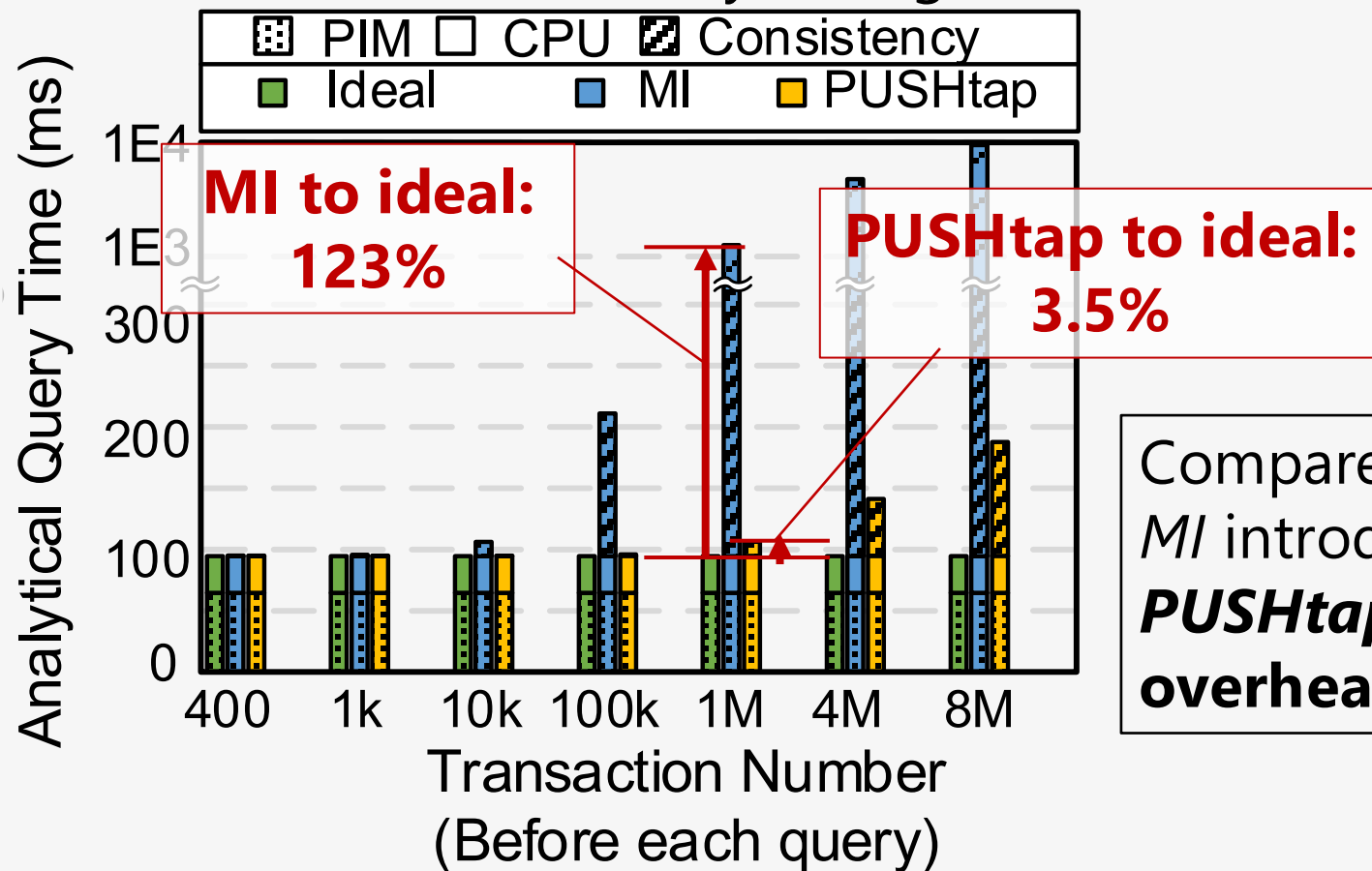
Result: Standalone Performance -- OLAP



OLAP Performance:

ideal: with zero-latency defragmentation

Consistency includes
defragmentation/
rebuilding



Compared to *ideal*: with 1M transactions, *MI* introduce 123% rebuilding overhead
***PUSHtap* only incurs 1.5% software overhead**



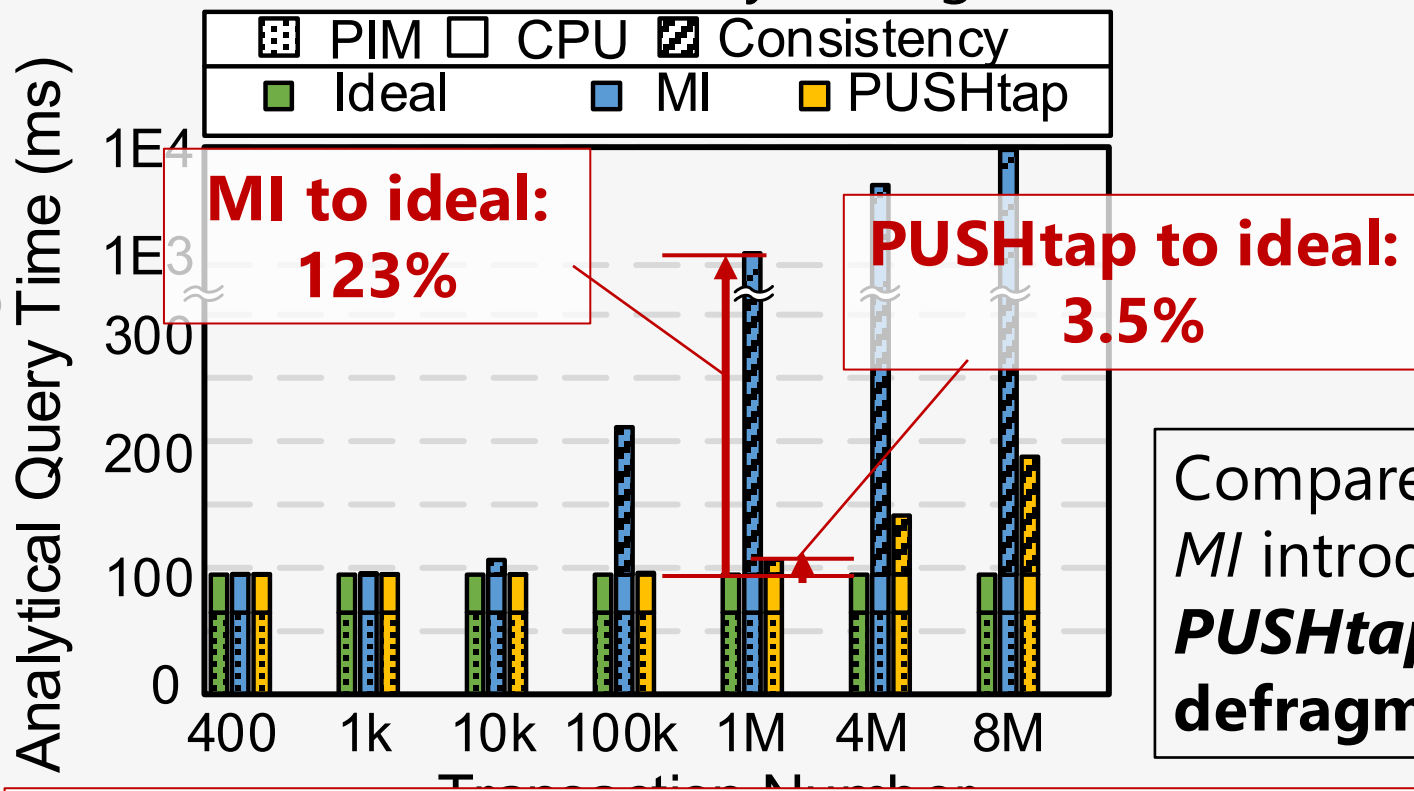
Result: Standalone Performance -- OLAP



OLAP Performance:

ideal: with zero-latency defragmentation

Consistency includes
defragmentation/
rebuilding



Compared to *ideal*: with 1M transactions,
MI introduce 123% rebuilding overhead
***PUSHtap* only incurs 1.5%
defragmentation overhead**

**PUSHtap performance is closed to ideal on both OLTP and OLAP
-- Workload-specific optimization**



[ISCA'24] UM-PIM: DRAM-based PIM with Uniform and Shared Memory Space

Yilong Zhao, Mingyu Gao, Fangxin Liu*, Yiwei Hu, Zongwu Wang, Han Lin, Ji Li, He Xian, Hanlin Dong, Tao Yang, Naifeng Jing, Xiaoyao Liang, Li Jiang*

Shanghai Jiao Tong University, Tsinghua University and Huawei

[ASPLOS'25] PUSHtap: PIM-based In-Memory HTAP with Unified Data Storage Format

Yilong Zhao, Mingyu Gao, Huanchen Zhang, Fangxin Liu*, Gongye Chen, He Xian, Haibing Guan, and Li Jiang*

Shanghai Jiao Tong University, Tsinghua University

[ISCA'26] COSM: A Cooperative Scheduling Framework for Concurrent PIM and CPU Execution on Mobile Devices

Yilong Zhao=, Fangxin Liu=, Onur Mutlu, Mingyu Gao, Jian Liu, Haibing Guan, and Li Jiang*

Shanghai Jiao Tong University, ETH Zurich, Tsinghua University and Beihang University

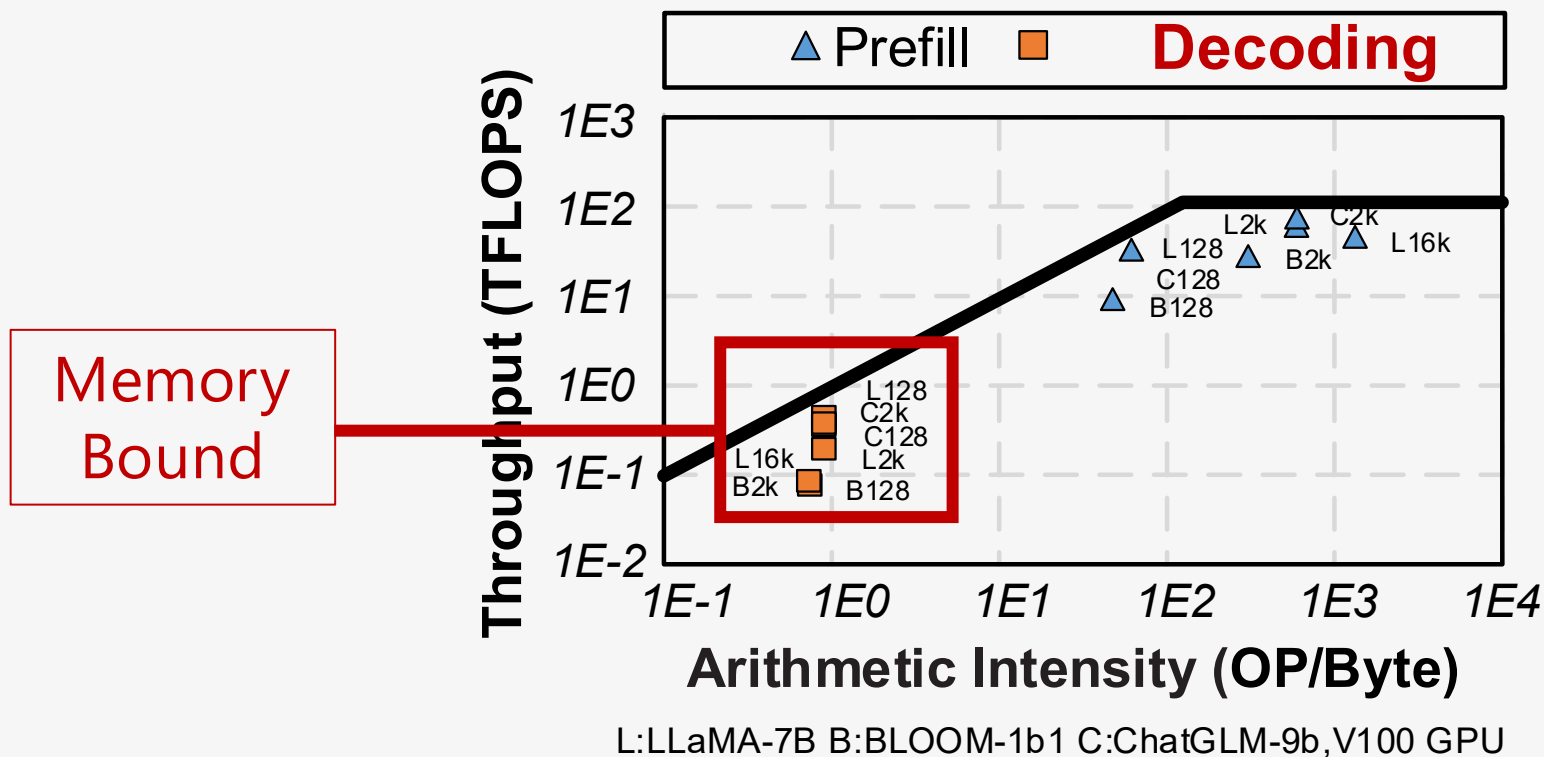
Monday, Jun 29 10:15 – 11:55 AM
Ballroom A, Session 1A



LLM Decoding – Memory Bound



LLM Decoding is Memory-bound:



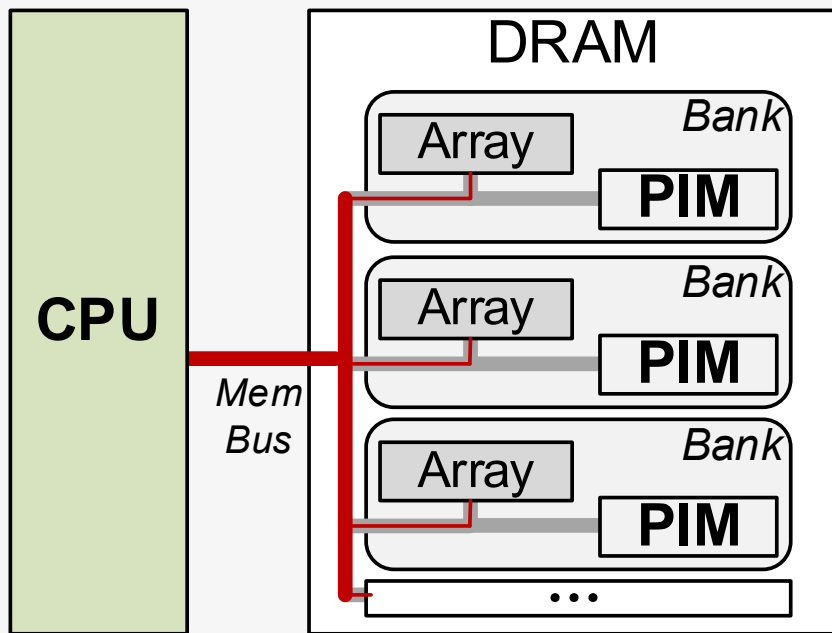
LLM decoding is inefficient on NPU/GPU because of limited bandwidth! → **Fit for PIM**



PIM & CPU Workload: Inefficient Bandwidth Use

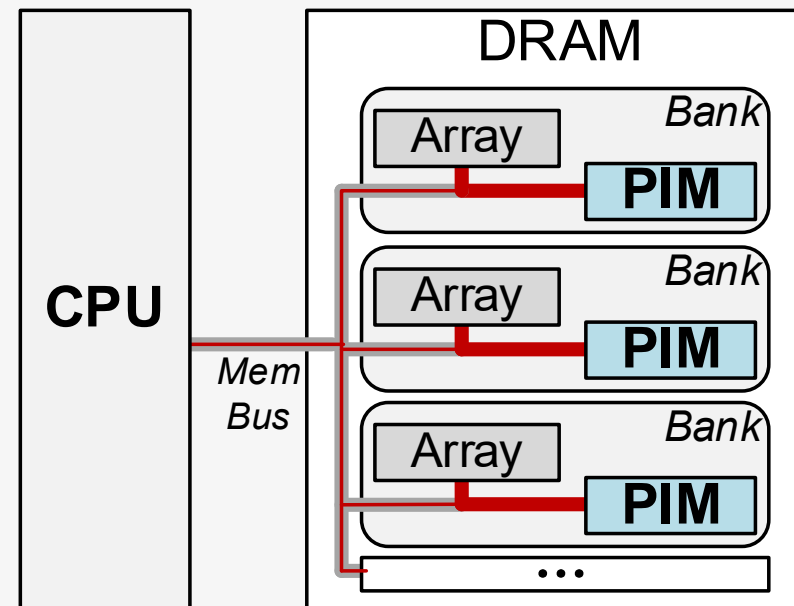


External bandwidth: memory bus bandwidth | **Internal bandwidth:** aggregated bank bandwidth



CPU Workload:
Only accesses

High **External Bandwidth** ↑
Low **Internal Bandwidth** ↓



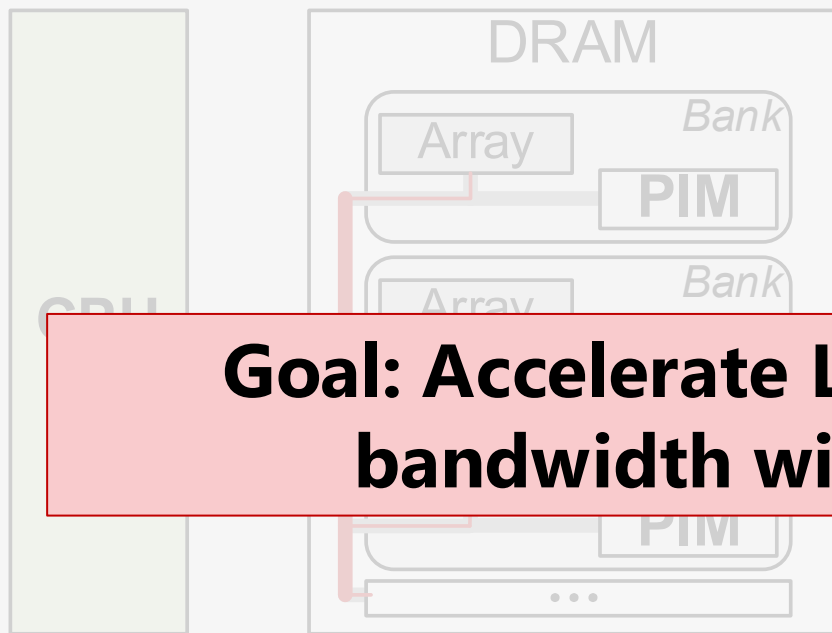
PIM Workload:
A few accesses + Massive executions

Low **External Bandwidth** ↓
High **Internal Bandwidth** ↑



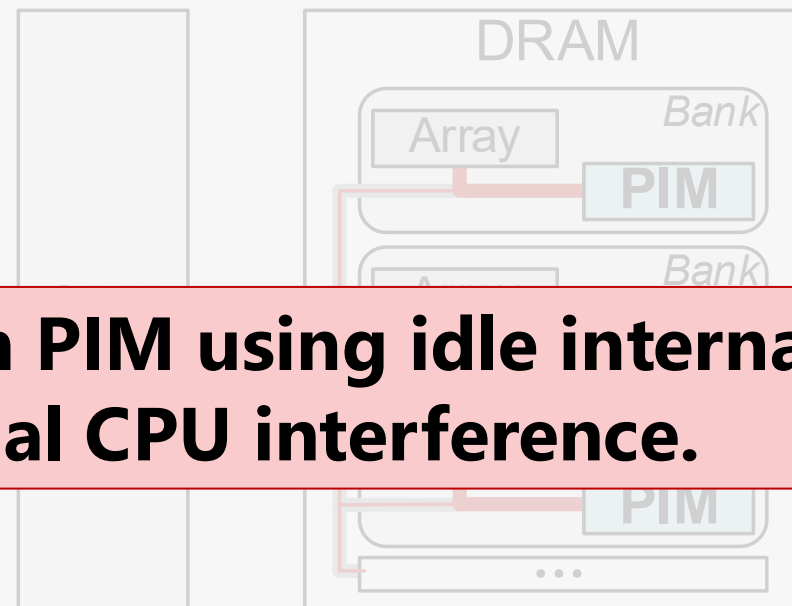
PIM & CPU Workload: Inefficient Bandwidth Use

External bandwidth: memory bus bandwidth | **Internal bandwidth:** aggregated bank bandwidth



CPU Workload:
Only accesses

High **External Bandwidth** ↑
Low **Internal Bandwidth** ↓



PIM Workload:
A few accesses + Massive executions

Low **External Bandwidth** ↓
High **Internal Bandwidth** ↑

Goal: Accelerate LLMs with PIM using idle internal bandwidth with minimal CPU interference.



COSM: A Cooperative Scheduling Framework



Objective: **Co-optimize Interface & Scheduling**

Challenge 1: The dilemma of fixed-length commands

Challenge 2: PIM data transfers starve CPU access

Challenge 3: CPU & PIM performance

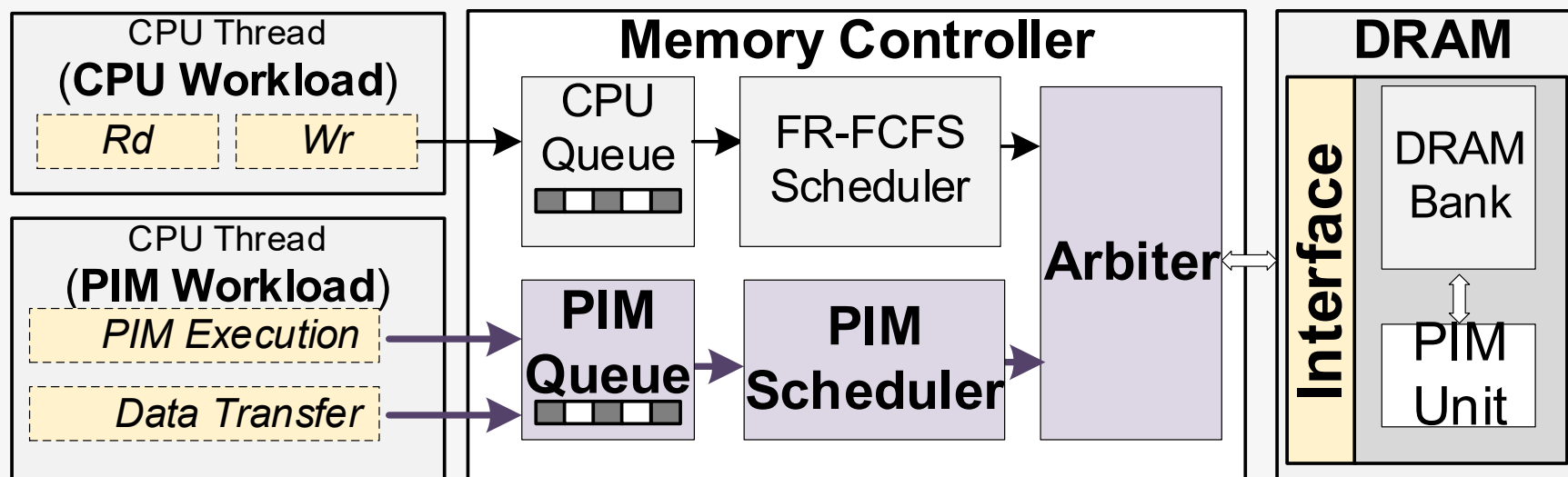
Solution :

Low-interference Interface

→ Preemptable PIM Execution

→ Bandwidth-decoupled Data Transfer

Idleness-aware Memory Scheduling





COSM: A Cooperative Scheduling Framework

Objective: **Co-optimize Interface & Scheduling**

Challenge 1: The dilemma of fixed-length commands

Challenge 2: PIM data transfers starve CPU access

Challenge 3: CPU & PIM performance

Solution :

Low-interference Interface

→ Preemptable PIM Execution

→ Bandwidth-decoupled Data Transfer

Idleness-aware Memory Scheduling

[ISCA'26] COSM: A Cooperative Scheduling Framework for Concurrent PIM and CPU Execution on Mobile Devices

Yilong Zhao⁼, Fangxin Liu⁼, Onur Mutlu, Mingyu Gao, Jian Liu, Haibing Guan, and Li Jiang^{*}

Shanghai Jiao Tong University, ETH Zurich, Tsinghua University and Beihang University

Monday, Jun 29 10:15 – 11:55 AM
Ballroom A, Session 1A



上海交通大學

SHANGHAI JIAO TONG UNIVERSITY

Thank you!

飲水思源 愛國榮校

Efficient Memory Management and Access Mechanisms for CPU-PIM Co-Execution Systems

Yilong Zhao

Shanghai Jiao Tong University

June 21, 2026

饮水思源 · 爱国荣校